

Návod k softwaru HeatBattery

simulace tepelných akumulátorů

Tým HeatBattery

7. května 2026

Tento dokument je český průvodce balíčkem softwaru **HiTepOptim** (`heat_battery`). Stručný anglický přehled najdete v kořenovém souboru [README.md](#); tento návod jde více do hloubky a popisuje, jak se kód v repozitáři skládá a jak se používá v praxi.

Průběžná dokumentace v kódu. Většina tříd, metod a funkcí v softwaru má vlastní docstring. Moderní IDE (například VS Code / Cursor, PyCharm nebo Spyder) tyto docstringy načítá přímo při psaní kódu, zobrazuje je v náhledu a umožňuje rychle přejít na definici daného objektu. Pro konkrétní argumenty funkcí, návratové hodnoty a aktuální chování API je to často nejpřesnější zdroj dokumentace v daném okamžiku.

```
(function) def save_mesh_add_data(
    add_data_file_path: str | Path,
    call_data: dict[str, Any],
    dim: int,
    points: dict[str, dict[str, list[float]]],
    mats: dict[str, tuple[str, list[int]]],
    bcs: dict[str, list[int]],
    jac_f: () -> float,
    custom_data: dict | None = None
) -> None

Saves mesh metadata to binary file.

Args:
    add_data_file_path: Path to the file to save the metadata to
    call_data: Dictionary containing the call data
    dim: Dimension of the mesh (2, 3)
    points: Dictionary containing the points (X, Y, Z) coordinates
    mats: Dictionary containing the volume materials groups
    bcs: Dictionary containing the surface boundaries groups

Examples:
>>> save_mesh_add_data(
>>>     add_data_file_path='mesh_add_data.ad',
>>>     call_data={'diameter': 1.0, 'height': 1.0},
>>>     dim=2,
>>>     points={'T': {'Tmid_1': [0.0, 0.0, 0.0]}},
>>>     mats={'Sand': (heat_battery.materials.Sand, [1])},
>>>     bcs={'outer_surface': [1]},
>>>     jac_f=lambda x: 2*pi*x[0],
>>>     custom_data={'master_key': {'subkey': 'value'}})
```

Docstring jako kontextová nápověda přímo při práci s kódem.

Obsah

1	Účel a architektura	2
2	Požadavky a instalace	4
3	Rychlý start	5

4	Konfigurace (<code>config.yaml</code>)	6
5	Matematický model a slabá formulace	7
6	Práce se simulací (<code>Simulation</code>)	8
7	Termy, ko-simulace a řízení	11
7.1	Tutoriál: <code>Example_01</code> krok za krokem	12
7.2	Termy zevnitř – jak fungují <code>UniformHeatSource</code> a <code>AmbientCooling</code>	16
8	Sondy, checkpointy a výstupy	19
9	Meteorologie a měření	22
10	Parametrické studie (<code>ParameterGrid</code>)	22
11	PostgreSQL projekt, fronta úloh a worker	23
12	Vizualizace	25
13	Přehled příkladů (<code>examples/</code>)	26
14	Optimalizace a korektnost úlohy	31
15	Ověření funkčnosti softwaru	32
16	Časté problémy	40

1 Účel a architektura

Představení softwaru

Nástroj **HiTepOptim** (v repozitáři jako balíček `heat_battery`) je soubor navazujících softwarových komponent pro definici, simulaci, systematické studie a vizualizaci výsledků systémů využívajících vysokoteplotní akumulaci tepla ve formě **citelného tepla**. Jeho základem je přednastavený MKP řešič vedení tepla, který umožňuje sledovat detailní prostorové rozložení teploty v akumulátoru.

Akumulátor obvykle nevystupuje jako izolovaný objekt, ale jako část většího energetického systému. S okolním modelem, často popsaným jednodušším 1D přístupem, si vyměňuje energii přes okrajové podmínky (například předávání tepla povrchem) nebo přes objemové zdrojové členy (například elektrický odporový ohřev topnými patronami). Geometrie akumulátoru proto ovlivňuje nejen celkovou kapacitu, ale i dynamiku maximálního nabíjecího a vybíjecího výkonu. Nevhodné uspořádání může omezit výkon i tehdy, když je akumulátor z hlediska celkové energie nabitý, nebo může vést k lokálnímu přehřívání topidel.

Simulace několikaletého provozu vytvářejí velké objemy dat, zejména časové řady sond s jemným časovým krokem. U parametrických studií tento objem dále roste. Součástí nástroje je proto i databázové zázemí a webové rozhraní pro interaktivní prohlížení výsledků v grafech. Cílem není skrýt fyzikální podstatu modelu, ale omezit množství rutinních kroků spojených se sestavením, stabilizací a vyhodnocením MKP výpočtu.

Software obsahuje několik příkladů, které slouží k ověření instalace i jako vzor pro vlastní úpravy. Současná verze pracuje s omezeným výběrem materiálů a komponent navazujícího systému (například FVE, budova nebo PID řízení), ale způsob zadávání zůstává flexibilní díky Pythonu a připravenému vysokoúrovňovému rozhraní. Do budoucna se počítá s rozšířením komponent i vlastností teplotnosných médií, například

vzduchu a vody.

Hlavním účelem softwaru je poskytnout cestu k odbornému zhodnocení systémů akumulace energie ve formě citelného tepla v různých aplikačních kontextech (domácnosti, sídliště, průmysl nebo energetika). Nástroj je určený pro uživatele, kteří jsou schopni správně definovat fyzikální systém, ale nemají se při každé studii zabývat všemi technickými detaily MKP implementace. Není zamýšlen jako aplikace pro širokou veřejnost. Je sestaven výhradně z open-source komponent a instalační skript je připraven pro Ubuntu, včetně použití přes WSL na počítačích s Windows.

Modelový přístup

Model popisuje chování tepelného akumulátoru jako součásti širšího energetického systému. Uživatel zadává zejména geometrii akumulátoru, materiály, zdroje tepla, okrajové podmínky a vazby na zbytek systému. Software se skládá z několika částí:

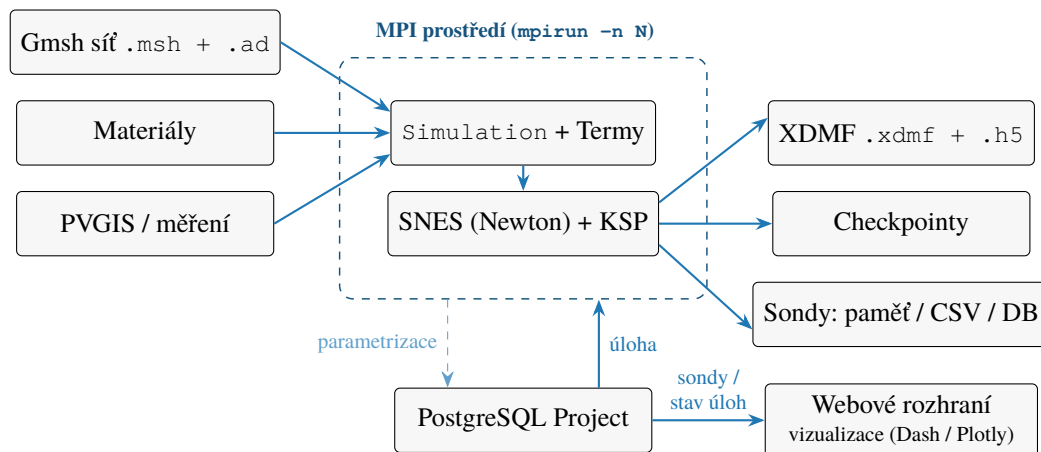
- simulačního jádra pro ko-simulaci akumulátoru (MKP) se zbytkem systému a pro numericky stabilní řízení časového výpočtu,
- sestavení MKP úlohy akumulátoru a jejích objemových i okrajových členů podle zadané geometrie,
- databázové vrstvy pro ukládání výsledků jednotlivých simulací a parametrických studií,
- webového rozhraní pro práci s databází, především pro vizualizaci výsledků.

Model akumulátoru

Výpočtový model akumulátoru je implementován jako Python knihovna. Jeho hlavním vstupem je geometrie, která může vznikat parametricky z Python funkce a následně se předává simulační třídě. Geometrická metadata obvykle obsahují:

- síť ve formátu Gmsh `.msh` s fyzikálními skupinami pro objemy a části hranice,
- argumenty funkce, která geometrii vygenerovala,
- dimenzionalitu úlohy, zpravidla 2D nebo 3D,
- body pro sondy, například pro záznam teplot,
- přiřazení materiálů k objemovým podoblastem,
- pojmenované části hranice pro okrajové podmínky,
- Jakobián pro symetrie nebo jinou úpravu integrační míry,
- další údaje vzniklé při generování geometrie, například délky nebo počty kanálů.

Schéma níže ukazuje zjednodušený tok dat od vstupů přes výpočetní jádro až k výstupům.



Cílová skupina: výzkumní pracovníci a studenti, kteří mají základní zkušenost s Pythonem a prostředím **Linux** (doporučené jsou Ubuntu 22.04 nebo 24.04). U distribuovaných studií se navíc předpokládá **PostgreSQL**, dostupné **MPI** v clusteru a síťový přístup k databázi z výpočetních uzlů.

2 Požadavky a instalace

Systém a závislosti

- **Operační systém:** Linux. Ve Windows použijte **WSL** s distribucí Ubuntu; další kroky jsou stejné jako v nativním Linuxu (viz odkazy v `README.md`).
- **Software:** Python 3.8 nebo novější, **DOLFINx / FEniCSx**, **PETSc**, **MPI**; pro generování sítě typicky **Gmsh**. Kompletní závislosti jsou uvedeny v `pyproject.toml`.
- **Volitelně:** **ParaView** pro prostorovou vizualizaci výstupů ve formátu XDMF.

Postup instalace

```
git clone https://github.com/LiborKudela/heat-battery.git
cd heat-battery
bash install_scripts/install_ubuntu.sh --help
```

Nejčastější varianty:

Výpočetní worker bez lokální databáze:

```
bash install_scripts/install_ubuntu.sh -y
```

Worker včetně PostgreSQL (v clusteru stačí jeden tento uzel):

```
bash install_scripts/install_ubuntu.sh -y -p --ppass HESLO --ppassc HESLO
```

Heslo si zvolte a zapište je do lokálního souboru `config.yaml` (podrobnosti v [section 4](#)). Tento soubor pak umístíte do pravocní složky jednotlivých pracovních uzlů clusteru (databázový uzel může být zároveň pracovním uzlem, ale nikoliv naopak). Pokud chcete nainstalovat software na OS Windows (10/11) - je jediným prověřeným způsobem cesta přes Windows Subsystem for Linux (WSL). Ideální je nainstalovat distribuci Ubuntu ve které pak proběhne instalace stejně jako na standardním Ubuntu systému.

Ověření instalace

```
python3 -c "import heat_battery; heat_battery.test_package()"
```

Tento příkaz otestuje několik předdefinovaných testů a odsimuluje velmi krátké simulace. Následně reportuje úspěšné dokončení nebo selhání těchto testů.

3 Rychlý start

Následují tři běžné způsoby práce podle toho, co chcete spustit.

A. Rychlá lokální simulace (jeden případ bez databáze)

Nejjednodušší varianta spouští `solve_unsteady` přímo, ukládá sondy do lokálních CSV souborů a případně zapisuje XDMF. Fronta úloh v PostgreSQL se v tomto režimu nepoužívá. Můžeme tak spustit například přiloženou ukázkou:

```
mpirun -n 4 python3 -m examples.Example_05.run_example_size5
```

Skript `examples/Example_05/run_example_size5.py` ukazuje načtení geometrie (s mezipamětí přes `CachedGeometryBuilder`), nastavení checkpointů do lokálního adresáře a spuštění jednoho samostatného výpočtu.

B. Rozsáhlá parametrická studie na clusteru (PostgreSQL a worker procesy)

Parametrická studie, fronta úloh a výpočetní workery vyžadují platný lokální soubor `config.yaml`. Obvykle stačí doplnit údaje pro připojení k PostgreSQL; ostatní parametry mají použitelné výchozí hodnoty v souboru `heat_battery/base_config.yaml`. Podrobnosti jsou v [section 4](#), zejména v [bloku o minimálním nastavení](#).

1. Naplnit frontu úloh v databázi (na jednom z worker uzlů)

```
python3 -m examples.Example_05.run_example
```

2. Spustit pracovní smyčku na jednom nebo více uzlech

```
mpirun -n 4 python3 worker.py --num-jobs 5 --project project_example_05
```

Poznámka k příkazu. `-n 4` spustí čtyři MPI procesy. Hodnotu `-n` volte podle počtu fyzických CPU jader dostupných na daném uzlu. Pro běh jedné simulace přes více uzlů je potřeba nastavit MPI spouštění přes seznam hostů (např. [Open MPI hostfile](#)) a zajistit sdílené úložiště dostupné ze všech uzlů (např. [NFS na Ubuntu Serveru](#)). Je však potřeba zohlednit overhead způsobený síťovou komunikací mezi uzly (někdy může být méně více).

3. Sledovat průběh přes webový dashboard

```
python3 examples/Example_05/view_project.py
```

4 Konfigurace (`config.yaml`)

Modul `heat_battery/config.py` skládá konfiguraci ze dvou zdrojů:

1. **Globální šablonu** ze souboru `heat_battery/base_config.yaml`.
2. **Lokální doplněk** `config.yaml` z aktuálního pracovního adresáře, pokud soubor existuje (konstanta `LOCAL_CONFIG_FILE_PATH`).

Hodnoty z lokálního souboru **přepisují** globální šablonu. Pokud lokální soubor obsahuje klíč, který v šabloně neexistuje, slučování konfigurace skončí chybou (`merge_configs`). Je to záměrná kontrola proti překlepům. Pro nestandardní umístění konfiguračních souborů slouží `setup_global_config_path` a `setup_local_config_path`.

Minimální lokální konfigurace bez databáze

Globální šablona `heat_battery/base_config.yaml` už obsahuje většinu nastavení a jedná se funční tovární výchozí nastavení. Výpočty nevyžadující PostgreSQL databázi si s tímto továrním nastavením bohatě vystačí (např. `run_example_size5` a většina `examples/Example_01--04`).

Minimální lokální konfigurace s PostgreSQL databází

Tovární nastavení je nutné rozšířit minimálně o přístupové údaje do PostgreSQL databáze a doporučuje se nastavit i uživatelské jméno pro snadnější určení vlastnictví zadání simulací v databázi. Níže jsou uvedeny parametry, kterých se tyto změny týkají.

Důležité položky `base_config.yaml`

- `user` (kořen konfigurace, nikoli `database.postgres.user`) — metadata o autorovi. I při hodnotách `null` lze úlohy do fronty zařazovat; údaj se ukládá ve sloupci `created_by`. Pro přehled a dohledatelnost autora ve webovém rozhraní jej doporučujeme vyplnit, (zejména pokud do jednoho projektu vkládá zadání více uživatelů). Údaje není potřeba vyplnit všechny.
- `local_temp_dir` (výchozí `heat_battery_temp`, který se umístí do `cwd` worker procesu) — adresář, do kterého worker na výpočetním uzlu rozbalí zdrojové soubory úlohy stažené z databáze. Na daném uzlu musí být zapisovatelný a musí mít dostatek volného místa.
- `database.use_engine` — musí zůstat `postgresql`; jiné databázové rozhraní není v tomto repositáři implementováno.
- `database.postgres` — parametry připojení knihovny `psycopg2`.
 - `db_name` — název databáze PostgreSQL, v níž aplikace ukládá svá data (např. `heat_battery`).
 - `user` — obvykle `postgres`, pokud instalaci provedl dodávaný instalační skript; jinak účet přidělený správcem databázového serveru.
 - `password` — heslo zvolené při instalaci skriptem nebo heslo od správce serveru.
 - `host` — typicky veřejná IP adresa počítače, na němž je PostgreSQL nainstalován a zpřístupněn; při běhu na téže stroji často `localhost`.

- port — číslo TCP portu (celé číslo); standardní instalace používá 5432. Při nestandardním čísle portu uveďte např. 5433.

Úplný obsah `heat_battery/base_config.yaml`

Níže je kompletní obsah globální šablony z repozitáře. Lokální `config.yaml` k ní přidává pouze přepsání vybraných klíčů.

```
user:
  first_name: null
  last_name: null
  github_username: null
  email: null
  x_handle: null
  avatar_url: null
  link: null
  force_avatar_key: null

local_temp_dir: heat_battery_temp

database:
  use_engine: postgresql

  postgres:
    db_name: null
    user: null
    password: null
    host: null
    port: 5432
    connection_method: threaded_pool
    min_pool_size: 1
    max_pool_size: 10
    only_rank0_send_queries: true
    verbosity: 0
    bin_files_folder_prefix: null

    # Features that are disabled below, prevent the query in question from
    # being sent to the database from the python API. Next to the primary
    # safety mechanism of having the database administrator set the
    # appropriate grants, this is another layer of safety to prevent
    # accidental data loss.
    allow_create_database: true
    allow_create_schema: true
    allow_create_table: true
    allow_drop_database: false
    allow_drop_schema: true
    allow_drop_schema_cascade: true
    allow_drop_table: false
```

5 Matematický model a slabá formulace

Model řeší **MKP úlohu vedení tepla** na neuspořádané síti: geometrie a fyzikální skupiny vznikají v Gmsh, vlastní výpočet běží v DOLFINx. Časová diskretizace používá θ -schéma; při výchozí hodnotě `theta = 0.5` odpovídá metodě **Crank--Nicolson**. Nelineární část každého časového kroku řeší SNES z PETSc, lineární podsystémy pak KSP.

Slabá formulace (v souladu s implementací)

Nechť T je teplotní pole na nové časové hladině, T_n pole z předchozího kroku, v libovolná testovací funkce a J Jakobián souřadnic (v kódu `self.jac`). Sumy probíhají přes materiálové podoblasti Ω_j a pojmenované části hranice Γ . Pro $\theta = 0.5$ platí:

$$\begin{aligned}\mathcal{R}(T; v) = & \sum_j \int_{\Omega_j} \left[\rho(T) c_p(T) T - \rho(T_n) c_p(T_n) T_n \right] v J \, dx \\ & + \theta \Delta t \sum_j \int_{\Omega_j} \left[k(T) \nabla T \cdot \nabla v - q_{\text{vol}}(T, \mathbf{x}, t) v \right] J \, dx \\ & + (1 - \theta) \Delta t \sum_j \int_{\Omega_j} \left[k(T_n) \nabla T_n \cdot \nabla v - q_{\text{vol}}(T_n, \mathbf{x}, t_n) v \right] J \, dx \\ & + \theta \Delta t \sum_{\Gamma} \int_{\Gamma} g_{\text{bc}}(T, \mathbf{x}, t) v J \, ds \\ & + (1 - \theta) \Delta t \sum_{\Gamma} \int_{\Gamma} g_{\text{bc}}(T_n, \mathbf{x}, t_n) v J \, ds = 0 \quad \forall v.\end{aligned}$$

Slovně: reziduum kombinuje časovou změnu tepelného stavu, přibližně daného entalpií $\rho c_p T$, s θ -váženým příspěvkem nelineární **difuze** $k(T) \nabla T$, **objemových zdrojů** q_{vol} a **okrajových toků** g_{bc} . Objemové zdroje přicházejí přes termy `q_source` a ve slabé formulaci vystupují se znaménkem $-$; okrajové členy přicházejí přes `bcs_terms` se znaménkem $+$.

Poznámka. V budoucí verzi bude v časovém členu slabé formulace použita přímo **entalpie** místo současného modelování skrze výraz $\rho c_p T$.

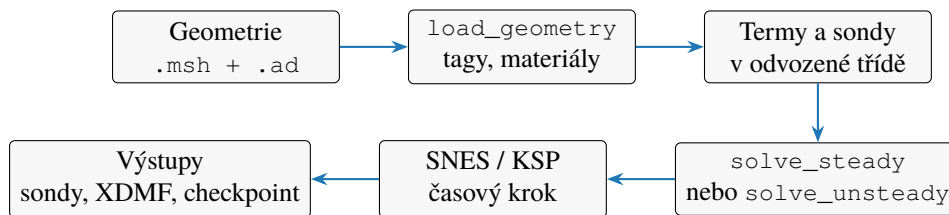
Nastavení tolerancí, časové integrace, výstupů a doplňkových voleb numerického řešiče (včetně režimu `solve_steady`) je popsáno v [section 6](#).

6 Práce se simulací (Simulation)

`Simulation` je základní třída pro vlastní model akumulátoru. V běžném použití ji uživatel neinstancuje přímo, ale vytvoří z ní odvozenou třídu, která popíše konkrétní fyziku dané úlohy: zdroje tepla, okrajové podmínky, sondy a uživatelsky pohodlné rozhraní pro spuštění výpočtu. Příklady v repozitáři tuto strukturu používají důsledně; typicky stačí převzít jeden z nich a upravit geometrii, materiály a několik metod modelové třídy.

Z praktického pohledu jsou při práci se simulací důležité čtyři kroky:

1. **Připravit geometrii a metadata** – adresář `geometry_dir` musí obsahovat soubor `.msh` a odpovídající `.ad` soubor. Název modelu v `model_name` musí odpovídat názvu těchto souborů bez přípony.
2. **Odvodit modelovou třídu ze `Simulation`** – v ní se napojí termy na pojmenované objemy a části hranice a nadefinují se sondy.
3. **Rozhodnout, co se má počítat** – pro rovnovážný stav slouží `solve_steady`, pro časový průběh `solve_unsteady`.
4. **Nastavit výstupy** – podle potřeby se zapínají CSV/DB sondy, XDMF pole pro ParaView a checkpointy pro pokračování přerušného běhu.



Zjednodušený životní cyklus modelu od geometrie po výstupy.

Životní cyklus konstruktora

Při vytvoření objektu modelu proběhne většina technické přípravy automaticky. Uživatel obvykle potřebuje vědět hlavně to, že názvy použité v kódu musí sedět na názvy fyzikálních skupin uložené v geometrii. Po zadání `geometry_dir` a `model_name` konstruktora:

1. `load_geometry` načte `.msh` a `.ad` metadata a vytvoří `domain`, `cell_tags`, `facet_tags`, materiálové objekty a mapy názvů podoblastí.
2. Vytvoří integrační míry (`dx`, `ds`, `dS`), funkční prostory a teplotní pole potřebná pro MKP výpočet.
3. Spočítá objemy materiálových podoblastí a plochy hraničních částí. Tyto hodnoty se používají například při normalizaci zdrojů a sond.
4. Zavolá `define_form_subdomain_terms`, kde odvozený model připojí zdroje a okrajové podmínky přes `set_source_term` a `set_bc_term`.
5. Podle nastavení argumentu `build_solvers` sestaví potřebné řešiče. Pokud víte, že budete spouštět pouze časový výpočet, není nutné sestavovat všechny varianty.

Co se typicky přepisuje

V jednoduchém modelu obvykle stačí přepsat dvě až čtyři metody:

- `define_form_subdomain_terms` – přiřadí zdroje a okrajové podmínky k pojmenovaným částem geometrie.
- `create_common_probes` – zaregistruje veličiny, které chcete sledovat v čase, například teploty v bodech nebo celkovou energii.
- `solve_steady` – volitelný obal pro stacionární výpočet. Hodí se, pokud chcete uživateli nabídnout několik pojmenovaných parametrů místo ručního nastavování konstant v termech.
- `solve_unsteady` – obdobný obal pro časový výpočet. Typicky převádí funkce času z run-skriptu na aktualizací callbacky termů.

Podrobnější ukázka těchto přepisů je v tutoriálu `Example_01` v následující kapitole. Při tvorbě nového modelu je praktické nejprve zprovoznit minimální stacionární nebo krátký nestacionární výpočet, teprve potom přidávat checkpointy, XDMF výstupy a delší časové horizonty.

Volby numerického řešiče

Slabá formulace se při běhu sestaví a předá nelineárnímu řešiči SNES a lineárnímu řešiči KSP z PETSc. V praxi se k numerickým volbám sahá hlavně ve chvíli, kdy výpočet konverguje pomalu, časový krok se příliš zmenšuje nebo se model chová výrazně nelineárně. Úplný seznam provozních parametrů, včetně časových kroků a výstupů, je v docstringách metod a v tabulce v [následující podsekcí](#).

Tolerance SNES a KSP

Parametry `atol`, `rtol`, `stol` a volitelně `ksp_rtol` určují kritéria ukončení iterací SNES a přidruženého KSP. Používají se u stacionárního i nestacionárního řešení. Při prvních pokusech je vhodné začít s výchozími hodnotami a měnit je až ve chvíli, kdy solver opakovaně selhává nebo je zbytečně pomalý.

Vnější iterace nad Termy

Některé objekty `Term` mohou při ko-simulaci potřebovat uvnitř jednoho časového kroku dodatečné vnější iterace. Parametr `max_term_its` nastavuje jejich horní mez. Přepínač `force_explicit_terms` tuto smyčku vypne; příslušné termy se pak vyhodnotí pouze jednou na začátku kroku. Takový režim je vhodný jen tam, kde to dává fyzikální smysl a kde nevádí explicitní vazba mezi akumulátorem a okolním systémem (to je tam kde termy ovlivňují akumulátor, ale akumulátor neovlivňuje termy).

Projekce teploty v čase

Volba `use_time_projection` zapíná lineární extrapolaci pole T z předchozích časových hladin jako počáteční odhad pro Newtonovu metodu; typicky snižuje počet nelineárních iterací. U prudkých změn řízení nebo okrajových podmínek je ale vhodné sledovat, zda projekce nezhoršuje počáteční odhad.

Poznámka k dalšímu vývoji. Budoucí verze bude mít pokročilejší systém explicitního odhadu řešení pro nový časový krok, například extrapolaci vyššího řádu (kubickou apod.).

Stacionární režim

Metoda `solve_steady` řeší stejnou rodinu Termů bez časové diskretizace a slouží k výpočtu rovnovážného teplotního pole. Prakticky se hodí pro kontrolu geometrie, materiálů a znamének okrajových podmínek před spuštěním dlouhé časové simulace.

`solve_unsteady` – parametry pro běžné použití

Čas je v sekundách **simulovaného modelu** (ne reálného běhu). Pro první ověření modelu volte krátký `t_max`, častější výpis sond a raději menší `dt_max`; pro produkční běh pak nastavte výstupy tak, aby objem dat odpovídal tomu, co opravdu potřebujete vyhodnocovat.

Parametr	Význam
<code>T0</code> , <code>T_guess</code> , <code>h0_T_ref</code>	Počáteční teplota / Newtonův odhad / referenční teplota pro entalpii.
<code>t_max</code>	Konec simulace v simulovaném čase.
<code>dt_start</code> , <code>dt_min</code> , <code>dt_max</code>	Počáteční krok a jeho meze; určují kompromis mezi stabilitou a rychlostí.
<code>dt_xdmf</code>	Jak často zapsat plné pole T do XDMF; malé hodnoty rychle zvětšují data.
<code>dt_ctrl_interval</code>	Pásmo (dolní, horní) pro řízení kroku podle $\max \Delta T $.
<code>atol</code> , <code>rtol</code> , <code>stol</code> , <code>ksp_rtol</code>	Tolerance SNES / KSP.
<code>verbose</code>	Detailní výpis průběhu; při dlouhých bězích bývá praktičtější stručný režim.
<code>xdmf_file</code> , <code>result_dir</code>	Prostorový výstup (oba musí být zadane společně).
<code>probe_destinations</code> , <code>probes_callbacks</code>	Kam streamovat skalární sondy (paměť / CSV / DB) a callback po každém vyhodnocení.

Parametr	Význam
checkpoint_dir, checkpoint_dt, checkpoint_callbacks	Periodické uložení stavu pro pokračování po přerušení výpočtu.
load_initial_checkpoint	Cesta k existujícímu checkpoint adresáři pro pokračování.
datetime_start	Kotva časové značky sond ('YYYY-MM-DD HH:MM:SS.s', UTC).
call_backs, call_back_each_t, call_back_each_step	Periodicita uživatelských callbacků (čas / krok).

Úplný seznam argumentů a aktuální komentáře jsou v **docstringu solve_unsteady** přímo v kódu.

MPI

Kód předpokládá **distribuovanou síť** a používá `MPI.COMM_WORLD`. I lokální výpočet je proto vhodné spouštět přes `mpirun`. Počet procesů volte podle velikosti sítě, dostupné paměti a počtu fyzických jader. Příliš mnoho procesů nemusí výpočet zrychlit, protože roste režie komunikace mezi ranky. Textové výpisy směrujte přes `Simulation.print_r0` nebo `print_rank_0`; obě varianty vypisují pouze z ranku 0.

Lokální spuštění může vypadat například takto:

```
mpirun -n 8 python3 vas_skript.py
```

7 Termy, ko-simulace a řízení

Term (`heat_battery.simulations.terms`) je stavební prvek, kterým se do modelu přidává zdroj tepla, okrajová podmínka nebo řízená vazba na okolní systém. Při rozšiřování modelu je nejpraktičtější nejprve použít některý z existujících termů a pouze ho připojit k pojmenované části geometrie. Vlastní term má smysl psát až tehdy, když v knihovně chybí požadovaný fyzikální vztah, regulace nebo vazba na externí model.

Term se k modelu připojuje jedním ze dvou způsobů:

- **objemový člen** registrovaný přes `set_source_term(term, "název_oblasti")`,
- **okrajovou podmínku** registrovanou přes `set_bc_term(term, "název_okraje")`.

Pro běžné použití jsou nejdůležitější vstupy, které term dostává z run-skriptu: například výkon patrony, teplota okolí, součinitel přestupu tepla nebo signál regulátoru. Mohou to být konstanty i časově proměnné funkce. Simulace si callbacky termů vyhledá automaticky přes metody z rodiny `resolve_form_terms*_callbacks` (viz `simulation_base.py`). Většina callbacků je vnitřní mechanismus knihovny a uživatel je běžně nemění. Prakticky důležité jsou hlavně tyto dva:

- **converged** – vrací, zda je pro aktuální krok (resp. vnitřní iteraci kroku) splněna podmínka konvergence termu. U regulátorů nebo jiných implicitních vazeb tím určíte, kdy už je vazba mezi termem, teplotním polem a případnou ko-simulací dostatečně konzistentní.
- **adaptation** – vrací *faktor* na úpravu časového kroku Δt . Běhová logika vezme minimum přes všechny registrované adaptace, takže prudká změna například na výstupu regulátoru může automaticky zmenšit časový krok. Prakticky tím simulace přizpůsobí krok nejcitlivější části propojeného systému.

Ko-simulace zde znamená, že akumulátor neběží izolovaně. Například model budovy, FVE nebo regulátor mění vstupy termů v čase. Metoda `solve_unsteady` drží společnou časovou osu a podle potřeby spouští **vnější iterace** uvnitř jednoho kroku (`max_term_its`). Pokud je vazba jednosměrná, například předem známý výkon zdroje, obvykle stačí jednoduché časové funkce nastavené přes `set_update`; v takovém případě může být vhodné použít `force_explicit_terms`. Pokud term reaguje na aktuální teplotu akumulátoru, je třeba sledovat konvergenci a případně zmenšovat časový krok.

7.1 Tutoriál: Example_01 krok za krokem

Example_01 je nejmenší kompletní simulace v repozitáři a dobře poslouží jako výchozí šablona pro vlastní model. Popisuje **topný element ve válci s pískem**, uzavřený v izolované nádobě a ochlazovaný konvekcí po obvodu. Ukazuje celý minimální postup: vytvoření geometrie, odvození třídy `Simulation`, připojení termů, registraci sond a spuštění výpočtu bez databáze. Celý příklad se vejde do tří souborů:

```
examples/Example_01/
|-- geometry.py      # Gmsh skript -> .msh + .ad metadata
|-- model.py         # odvozena trida Simulation: Termy, sondy, spusteni
|-- run_example.py   # vygeneruje sit, instancuje model, spusti solve_unsteady
```

Krok 1: Geometrie a metadata

Geometrie Example_01 odpovídá laboratornímu uspořádání: válec s pískem, ocelové stěny, izolace a topná patrona. Pro další práci je podstatné, aby geometrie měla pojmenované materiálové oblasti a pojmenované části hranice. Stejnými názvy se potom odkazujete na oblasti v modelové třídě. V Gmsh ji skládá skript `geometry.py` a ukládá dva výstupy:

- `mesh_2d.msh` – síť s **pojmenovanými fyzikálními skupinami** pro jednotlivé materiály ('sand', 'insulation', 'heated cartridge', ...) a pro okrajovou část 'outer_surface'.
- `mesh_2d.ad` – binární „metadata“ slovník, který `Simulation.load_geometry` načítá automaticky:

API Gmsh tento návod nerozebírá. Pokud potřebujete geometrii v Gmsh stavět nebo upravit detailněji, použijte [dokumentaci Gmsh](#). Pro HeatBattery je důležité hlavně to, co má být ve výstupním `.msh` souboru (pojmenované skupiny, angl. *Physical Groups*) a co má obsahovat slovník `add_data` (angl. *additional data*).

```
add_data = {
    'dim': 2,                                # dimenzionalita geometrie
    'materials': mats,                       # {jmeno: (Material_trida, [tagy_bunek])}
    'boundaries': bcs,                      # {jmeno: [tagy_okraje]}
    'jac_f': lambda x: 2*pi*x[0],          # faktor integrace podle objemu (Jakobian)
    'points': {'T': {...}},                 # body pro probe sampler
    'custom_data': {...},                  # libovolny balik vlastnich hodnot
}
```

Následují ilustrační příklady jednotlivých klíčů slovníku `add_data`:

- 'dim' – dimenze úlohy v HeatBattery smyslu, např. 2 u 2D axisymetrického řezu v rovině (r, z).

```
'dim': 2,
```

- 'materials' – mapa z názvu fyzikální skupiny v Gmsh na dvojici „třída materiálu + tagy buněk“ (viz ukázka níže). Klíče musí přesně odpovídat názvům `Physical Group` pro objem.

```
'materials': {
    'sand': (materials.SandTheory, [7]),
    'insulation': (materials.Standard_insulation, [2]),
    'heated cartridge': (materials.Cartridge_heated, [4]),
},
```

- 'boundaries' – mapa názvu okrajové části → seznam tagů křivek nebo ploch (Physical Group pro dim-1); stejné názvy se potom používají v `set_bc_term(...)`.

```
'boundaries': {
    'outer_surface': [13, 14, 15, 16, 17, 18, 33, 34],
},
```

- 'jac_f' – funkce souřadnice bodu x vracející faktor Jakobiánu (objemový element). Příklad: při *axisymetrii podél osy y* v řezu, kde radiální souřadnice odpovídá $x[0]$, je častá volba $2\pi r$. Pro jinou volbu os sítě musí být `jac_f` konzistentní s integrací v kódu.

```
'jac_f': lambda x: 2*pi*x[0], # r = x[0]; axisym. podle osy y
```

- 'points' – vnořený slovník: skupina sond (např. teplota 'T') a pod ní jméno → souřadnice ve stejném systému jako po zpracování v `geometry.py` (viz `create_common_probes`).

```
'points': {
    'T': {
        'teplota v topidle': [0.022, 0.15, 0.0],
        'teplota ve stredu': [0.0, 0.0, 0.0],
    },
},
```

- 'custom_data' – libovolný vlastní slovník; hodnoty čtete v modelu přes `self.geo_meta` (případně i jiné klíče, pokud je `load_geometry` propíše).

```
'custom_data': {
    'vlastní udaj': 'toto je ukazka',
    'hodnota z gmsh CADu': gmsh.model.getValue('physical_name', tag),
},
```

`materials` a `boundaries` se předávají třídě `Simulation`. Klíče těchto slovníků jsou pracovní názvy, které potom používáte v modelové třídě. Pokud se název v `set_source_term` nebo `set_bc_term` neshoduje s metadaty geometrie, model nelze správně sestavit.

Vlastní geometrie ve formátu `.step` / `.stl` se obvykle připojuje v `build_geometry`. Podobný postup je uveden v příkladech [Example_02](#) a [Example_04](#), z nichž lze při úpravách vycházet.

Krok 2: Modelová třída – Experiment_v1

```
# examples/Example_01/model.py
from heat_battery.simulations.simulation_base import MPI, Simulation, fem
from heat_battery.simulations.probing import FunctionSampler
from heat_battery.simulations import terms

class Experiment_v1(Simulation):

    def define_form_subdomain_terms(self):
        # objemovy zdroj v patrone
        self.set_source_term(terms.UniformHeatSource(self), "heated cartridge")
        # konvekce na vnějším povrchu uloziste
        self.set_bc_term(terms.AmbientCooling(self), "outer_surface")

    def create_common_probes(self, probes):
        # body z metadat -> sondy teploty
        coords = list(self.geo_meta["points"]["T"].values())
        sampler = FunctionSampler(coords, self.domain)

        @probes.register_probe("T", unit="degC")
        def Tc_probe():
            return sampler.eval(self.T)

        # sonda celkoveho tepla v domene
        h_form = sum(mat.h(self.T)*mat.rho(self.T)*self.jac*self.dx(i)
                     for i, mat in enumerate(self.mats, 1))
        H_form = fem.form(h_form)

        @probes.register_probe("heat", "J")
        def H_probe():
            H = fem.assemble_scalar(H_form)
            return self.domain.comm.allreduce(H, op=MPI.SUM)

    def solve_steady(self, Qc=10, T_amb=20, alpha=6.3, **kwargs):
        self.get_source_term("heated cartridge").set_steady_state_value('Qc', Qc)
        obj = self.get_bc_term("outer_surface")
        obj.set_steady_state_value('T_amb', T_amb)
        obj.set_steady_state_value('alpha', alpha)
        return super().solve_steady(**kwargs)

    def solve_unsteady(self,
                       Qc_t=lambda t: 100,
                       T_amb_t=lambda t: 10.0,
                       alpha_t=lambda t: 6.3,
                       **kwargs):
        # nastavit uzivatelske funkce do Termu
        self.get_source_term("heated cartridge").set_update('Qc', Qc_t)
        self.get_bc_term("outer_surface").set_update('T_amb', T_amb_t)
        self.get_bc_term("outer_surface").set_update('alpha', alpha_t)
        return super().solve_unsteady(**kwargs)
```

V Experiment_v1 jsou proti základní třídě Simulation přepsané právě ty části, které se při vlastním modelování obvykle mění. Základní třída dál řeší sestavení MKP problému, časovou integraci a výstupy:

define_form_subdomain_terms – V této metodě se fyzika modelu propojí s geometrií. Vyberou se termy a přiřadí se ke konkrétním názvům oblastí nebo částí hranice. Základní třída metodu zavolá po načtení sítě, kdy už zná materiálové oblasti i pojmenované části hranice. V tomto příkladu se zdroj tepla

přiřadí objemu "heated cartridge" a konvektivní chlazení okrajové části "outer_surface". Názvy musí odpovídat fyzikálním skupinám v .msh; konkrétní matematický tvar příspěvku určuje použitá třída termu.

create_common_probes – Tady se rozhoduje, co má výpočet průběžně ukládat. Do sond patří hlavně veličiny, podle kterých budete později hodnotit chování systému: teploty ve vybraných bodech, celková energie, výkon nebo diagnostika solveru. Díky tomu zůstává postprocessing součástí modelu a stejné sondy lze použít ve stacionárním i nestacionárním běhu. V tomto příkladu se z metadat geometrie načtou body teplotních sond. `FunctionSampler` v nich vyhodnocuje MKP řešení teplotního pole. Registrace sondy "T" vytvoří časové řady odpovídající jednotlivým bodům teplotních sond ve stupních Celsia (`degC`). Druhá sonda integruje přes materiálové díly výraz $h(T) \rho(T)$ s objemovým faktorem `jac` (Jakobián). Fyzikálně jde o energii uloženou v úložišti, tedy o *citelné teplo* vázané na teplotu materiálu. Příspěvky z jednotlivých MPI procesů se sečtou přes `allreduce MPI.SUM`, protože každý proces integruje jen svou část výpočetní sítě.

solve_steady – Tato metoda je praktické uživatelské rozhraní pro rovnovážný výpočet. Nejprve převezme vstupní argumenty z run-skriptu a přiřadí je příslušným termům, tedy zdrojovým členům a okrajovým podmínkám. Teprve potom spustí stacionární solver ze základní třídy `Simulation`. V tomto příkladu přijímá skalární hodnoty `Qc`, `T_amb` a `alpha`; před Newtonovou iterací je zapíše do příslušných termů přes `set_steady_state_value`. Ostatní argumenty, například adresáře výsledků nebo nastavení sond, předá základnímu řešiči beze změny. Formát těchto uživatelských argumentů není pevně daný knihovnou. Volí jej autor odvozeného modelu podle toho, jak chce model ovládat z run-skriptu.

solve_unsteady – Tato metoda plní stejnou roli pro časový výpočet. Nejprve převezme časově závislé vstupy z run-skriptu a přiřadí je příslušným termům, tedy zdrojovým členům a okrajovým podmínkám. Potom spustí nestacionární solver ze základní třídy `Simulation`, který už řídí časovou integraci, adaptaci kroku, sondy, XDMF výstupy a checkpointy. V tomto příkladu se funkce `Qc_t`, `T_amb_t` a `alpha_t` (výchozí `lambda` v podpisu metody nebo funkce předané z run-skriptu) připojí ke konstantám přes `set_update`. Argumenty `**kwargs`, například `t_max`, výstupy sond, XDMF nebo checkpointy, se beze změny předávají do `super().solve_unsteady(**kwargs)`. Také zde si tvar dodatečných argumentů, například názvy funkcí času a jejich výchozí hodnoty, určuje uživatel při rozšiřování vlastní modelové třídy.

Krok 3: Run-skript

Run-skript je nejvyšší vrstva konkrétní simulace. Připraví pracovní adresáře, vygeneruje nebo načte geometrii, vytvoří instanci modelové třídy a spustí zvolený režim výpočtu. Právě zde se zadávají uživatelské parametry běhu: časový horizont, průběhy výkonu nebo teploty okolí, umístění výsledků a nastavení sond. Díky tomu zůstává modelová třída opakovaně použitelná a jednotlivé scénáře se liší hlavně obsahem run-skriptu.

```
# examples/Example_01/run_example.py
from .geometry import build_geometry
from .model import Experiment_v1
import math, os

example_dir = os.path.dirname(__file__)
g_dir = os.path.join(example_dir, 'meshes')
r_dir = os.path.join(example_dir, 'results')

build_geometry(dim=2, dir=g_dir)          # vygeneruje mesh_2d.msh + .ad

sim = Experiment_v1(geometry_dir=g_dir, model_name='mesh_2d')

sim.solve_unsteady(
    verbose=True,
    t_max=100.0,
    Qc_t=lambda t: 100 + 10*math.sin(2*math.pi*0.01*t),
    T_amb_t=lambda t: 10.0,
    result_dir=r_dir,
    probe_destinations=[{
        'type': 'csv',
        'file_name': 'unsteady.csv',
        'result_dir': r_dir,
        'flush': True,
    }],
)

```

Spuštění bez PostgreSQL:

```
mpirun -n 4 python3 -m examples.Example_01.run_example
```

Soubor `examples/Example_01/results/unsteady.csv` obsahuje časový průběh všech sond, například $T[0]-T[14]$, `heat` a čítače typu `NLS_iter`.

7.2 Termy zevnitř – jak fungují `UniformHeatSource` a `AmbientCooling`

Při běžném použití většinou stačí existující term nastavit aktualizací funkcí nebo stacionární hodnotou. Pokud ale potřebujete nový typ zdroje, okrajové podmínky nebo regulace, je užitečné vědět, co term uvnitř dělá. Term není jen funkce vracující člen slabé formy. Je to objekt, který drží reference na hodnoty ovlivňující slabou formulaci v MKP, typicky přes **FEM konstanty** (`fem.Constant`). Díky tomu lze hodnoty ve formulaci měnit mezi časovými kroky bez opětovného sestavení celé formy. Ke každé takové hodnotě může term držet aktualizací funkci (`update`), kterou lze nastavit na libovolnou funkci času.

Anatomie Termu

```
# heat_battery/simulations/terms.py
class UniformHeatSource(Term):
    def define_terms(self):
        # [W], merný objemový zdroj
        self.new_constant('Qc', 0.0)
        # vychozi callback, který muzete prepsat
        self.set_update('Qc', lambda t: 0.0)
        # automaticky integrovany citac
        self.new_integral('Q_cumulative', 0.0, 'Qc')

    def __call__(self, T, x, t=None, domain=None):
        i = self.sim.subdomain_map[domain]
        Vc = self.sim.V_subdomain[i]
        # UFL vyraz, jednotky: [W/m^3]
        return self.get_constant('Qc', t) / Vc
```

Pro vlastní úpravy jsou v ukázce podstatné hlavně tyto části:

- `new_constant('Qc', 0.0)` vytvoří veličinu, která může být použita ve výrazu rovnice vstupující do MKP. Interně alokuje `fem.Constant` se **dvěma sloty**: `value[0]` pro nový čas t_{n+1} , `value[1]` pro starý čas t_n . Díky tomu může term přispět do **Crank--Nicolsonova** schématu na obou koncích časového kroku.
- `set_update('Qc', f)` nastaví časový průběh dané veličiny. Před řešením kroku se zavolá $f(t)$ a výsledek se uloží do správného slotu konstanty. f je běžná funkce času zapsaná v Pythonu; může být konstanta, interpolace dat, regulátor nebo vazba na jiný model. Někdy je výchozí `update` nastaven přímo uvnitř termu, jindy ho přepíše až odvozená metoda typu `solve_steady` nebo `solve_unsteady`.
- `__call__` určuje vlastní matematický příspěvek termu. Vrací **UFL výraz**, který se použije při sestavení příspěvku v samotné simulaci. Podle toho, zda je term registrovaný jako objemový nebo okrajový, se integruje přes `dx(i)` nebo `ds(i)`. Term se zároveň může odkazovat na svou instanci `Simulation` přes `self.sim`; tím lze využít například mapy podoblastí, objemy, plochy, materiály nebo jiné informace připravené základní třídou. V tomto případě vrací objemovou hustotu výkonu (W/m^3), takže $\int_{\Omega_j} q \, v \, dx$ dává $\text{W} \rightarrow \text{J/s} \rightarrow$ správný rozměr pro tepelnou rovnici.
- `converged()` je volitelný callback pro termy s implicitní vazbou na řešené teplotní pole. Základní `Term` ho nemá; pokud ho term nedefinuje, simulace předpokládá, že term neklade žádnou dodatečnou podmínku konvergence.
- `adaptation()` je volitelný callback pro úpravu časového kroku. Základní `Term` ho také nemá; pokud není definován, term časový krok nijak neomezuje. Používá se hlavně u regulátorů nebo členů, kde prudká změna vstupu vyžaduje jemnější časové rozlišení.

`AmbientCooling` je příklad okrajového termu. Z uživatelského pohledu jsou důležité hlavně dvě nastavitelné veličiny: teplota okolí `T_amb` a součinitel přestupu tepla `alpha`. Tělo termu je jednoduché:

```

class AmbientCooling(Term):
    def define_terms(self):
        self.new_constant('T_amb', 20.0)
        self.set_update('T_amb', lambda t: 20.0)
        self.new_constant('alpha', 6.3)
        self.set_update('alpha', lambda t: 6.3)

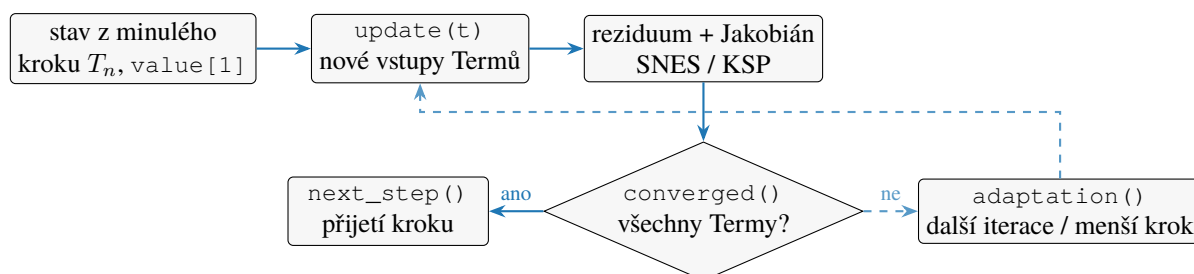
    def __call__(self, T, x, t=None, domain=None):
        return self.get_constant('alpha', t)*(T - self.get_constant('T_amb', t))

```

Jde o běžné **Newtonovo ochlazování** $g = \alpha(T - T_\infty)$, které se v slabé formě integruje přes $ds(i)$ na dané části hranice. Vlastní model pak už jen rozhodne, ke které pojmenované části hranice se tento term připojí. O vložení UFL výrazu do MKP formulace se postará třída *Simulation*.

Co se děje v jednom časovém kroku

Tento detail není nutné řešit při každém spuštění simulace, ale pomáhá při ladění regulátorů a vlastních termů. V jednom kroku se typicky děje:



Vnější iterace nad Termy v jednom časovém kroku.

1. Na začátku kroku už slot `value[1]` obsahuje hodnotu z předchozí přijaté časové hladiny. Tu do něj po minulém kroku přesunulo `next_step()`.
2. *Simulation* nastaví nový čas `sim.t` a zavolá aktualizace termů pro nový čas: `update(t = sim.t)` ($f(t) \rightarrow \text{value}[0]$).
3. *Simulation* sestaví reziduum a Jakobián a spustí SNES / KSP.
4. Po vyřešení nelineární úlohy je v objektu *Simulation* k dispozici nové teplotní pole T .
5. Simulace se zeptá termů přes `converged()`, zda jsou jejich interní vazby s novým teplotním polem dostatečně konzistentní.
6. Pokud všechny termy konvergují, simulace zavolá `next_step()`. Tím se aktuální hodnota `value[0]` přesune do starého slotu `value[1]`.
7. Pokud některý term nekonverguje, následuje další vnější iterace téhož časového kroku, nejvýše do limitu `max_term_its`.

Praktická pravidla:

- **Stacionární režim** používá oddělenou jednoslotovou konstantu (`fem_consts_steady[name]`), kterou nastavíte přes `set_steady_state_value(name, value)`. Ta se nemíchá s časovou variantou; proto lze mít pro stejný term jiné hodnoty ve stacionárním a nestacionárním běhu.

- `new_integral('Q_cumulative', 0, 'Qc')` automaticky vytvoří další konstantu, která v každém kroku integruje vybranou veličinu přes Δt v Crank--Nicolsonově schématu. Hodí se pro sondy typu „celková dodaná energie“.
- `prevent_override=True` u `set_update` chrání interní callback termu před přepsáním zvenčí. Hodí se u termů, které si hodnotu řídí samy, typicky u PID regulátoru.

Vlastní jednoduchý Term

Vlastní term píše tehdy, když požadovaný vztah nejde pohodlně vyjádřit pomocí existujících tříd. Minimální postup je jednoduchý: vytvořit třídu odvozenou od `Term`, v `define_terms` založit nastavitelné konstanty a v `__call__` vrátit UFL výraz. Příklad níže představuje konstantní tepelný tok na ploše po dobu $t < 60$ s:

```
from heat_battery.simulations.terms_base import Term

class PulseHeatFlux(Term):
    def define_terms(self):
        self.new_constant('q', 0.0)
        self.set_update('q', lambda t: 1000.0 if t < 60.0 else 0.0)

    def __call__(self, T, x, t=None, domain=None):
        return -self.get_constant('q', t) # vstupuje s minus -> chladi povrch
```

A v modelu:

```
self.set_bc_term(PulseHeatFlux(self), "outer_surface")
```

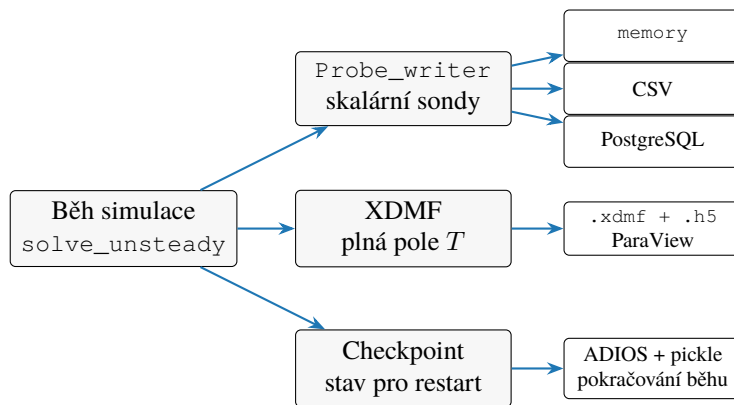
Tím do slabé formy přidáte **Neumannovu okrajovou podmínku** $-q$ v ds , aniž by bylo nutné zasahovat do jádra knihovny. Stejný vzor lze použít pro krátké pulzy, předpisy podle naměřených dat nebo jednoduché vazby na externí model.

Pro napojení **PID regulátoru** slouží `PIDTerm` (`terms_base.py`). Použijte ho tehdy, když akční zásah závisí na teplotě, která se teprve v daném kroku počítá. Term proto přidává callbacky `converged()` a `adaptation()`, aby se implicitní vazba mezi řízením a výslednou teplotou mohla iterovat a podle potřeby zmenšit časový krok (viz `Example_06`).

8 Sondy, checkpointy a výstupy

Sondy – `Probe_writer`

Sondy jsou pojmenované skalární veličiny vyhodnocované během výpočtu. Typicky jde o teploty ve vybraných bodech, celkové uložené teplo, výkon, čítače iterací nebo jiné diagnostické hodnoty. Slouží pro rychlé vyhodnocení průběhu simulace, protože není nutné kvůli každému grafu načítat celé prostorové pole. Kam se mají sondy ukládat, určuje seznam slovníků v argumentu `probe_destinations`:



Tři různé typy výstupů: rychlé skalární sondy, prostorová pole a restartovací stav.

type	Kam zapisuje
'memory'	Pandas DataFrame pouze v RAM. Nejde o trvalý záznam po ukončení procesu; hodí se hlavně pro testy a malé úlohy.
'csv'	Lokální CSV soubor. Používá se pro samostatné běhy bez databáze; cestu určují <code>result_dir</code> a <code>file_name</code> .
'database'	Uložení časových řad do PostgreSQL projektu. Hodí se pro delší běhy, porovnávání variant a parametrické studie (<code>result_database</code> , <code>project_name</code> , <code>table_name</code>).

Implementace je v [heat_battery/simulations/probing.py](#).

Ukázka definice sond v modelu

Sondy se obvykle definují v metodě `create_common_probes`. Tato metoda dostane objekt `probes`; pomocí dekorátoru `register_probe(...)` do něj přidáte funkce, které vracejí aktuální hodnotu sondy. V `Example_01` jsou použité dva typické případy: bodové teploty a integrální veličina přes celou doménu.

```

def create_common_probes(self, probes):
    # body z metadat geometrie -> sondy teploty
    coords = list(self.geo_meta["points"]["T"].values())
    sampler = FunctionSampler(coords, self.domain)

    @probes.register_probe("T", unit="degC")
    def T_probe():
        return sampler.eval(self.T)

    # integralni sonda celkove energie v ulozisti
    h_form = sum(mat.h(self.T)*mat.rho(self.T)*self.jac*self.dx(i)
                 for i, mat in enumerate(self.mats, 1))
    H_form = fem.form(h_form)

    @probes.register_probe("heat", "J")
    def H_probe():
        H = fem.assemble_scalar(H_form)
        return self.domain.comm.allreduce(H, op=MPI.SUM)

```

První sonda používá `FunctionSampler`. Souřadnice bodů se vezmou z metadat geometrie a při vyhodnocení sondy se v těchto bodech odečte aktuální teplotní pole `self.T`. Protože je sonda registrovaná pod

názvem "T", ve výstupním CSV vzniknou sloupce pro jednotlivé body teplotních sond, například $T[0]$, $T[1]$, ...

Druhá sonda ukazuje integrální veličinu. Nejprve se sestaví forma pro energii uloženou v materiálových částech domény a převede se na `fem.form`. Při každém vyhodnocení se lokální integrál spočítá přes `fem.assemble_scalar` a přes `allreduce` se sečtou příspěvky ze všech MPI procesů. Výsledkem je jedna skalární hodnota v joulech.

XDMF (plná pole)

XDMF výstup ukládá celé prostorové pole teploty T , tedy data vhodná pro ParaView a kontrolu rozložení teploty v akumulátoru. Zapíná se nastavením `xdmf_file` společně s `result_dir`. Při nestacionárním běhu se pole zapisuje každých `dt_xdmf` sekund simulovaného času; hodnoty se na požadované časy zápisu interpolují. Sít se zapíše pouze jednou na začátku.

XDMF je výrazně objemnější než sondy. Pro dlouhé výpočty proto ukládejte plná pole jen tak často, jak je skutečně potřebujete pro vizualizaci nebo kontrolu modelu. Pro běžné grafy časových průběhů jsou vhodnější sondy.

Checkpointy

Checkpoint slouží k pokračování přerušného výpočtu. Hodí se u dlouhých běhů, výpočtů na clusteru a parametrických studiích. Zapíná se nastavením `checkpoint_dt` a `checkpoint_dir`. V každé checkpointové značce se uloží:

- **konstanty času a entalpie** (`constants.pickle`),
- **funkce T , T_n** přes `adios4dolfinx` (`functions/`),
- **stavy termů** (`terms.pickle`),
- **stav cílů sond** (`probes.pickle`) – u CSV např. velikost souboru,
- **lokální proměnné cyklu** (`local_data.pickle`),
- **JSON metadata** (`metadata.json`),
- **Snapshot XDMF** dvojice `.xdmf + .h5` (pokud je XDMF zapnut) v podadresáři `xdmf/`.

Při spuštění s `load_initial_checkpoint=<dir>` se obnoví stav simulace z daného adresáře. Kromě teplotního pole se obnoví také interní stav termů, časové proměnné a stav výstupů. Prakticky to znamená mimo jiné:

- **CSV sondy** se zkrátí na uloženou délku, aby navazující zápis neponechal v souboru data z neplatné části přerušného běhu,
- **XDMF dvojice** se zkopíruje zpět a otevře se v režimu "a" (append) – pokud byl XDMF v původním běhu zapnut, animace v ParaView pokračuje plynule.

Nejbezpečnější je navazovat na checkpoint se stejným počtem MPI procesů, s jakým byl vytvořen. Stav teplotních polí se ale ukládá přes `adios4dolfinx` na původní síti, takže při nezměněné geometrii a dostupném sdíleném úložišti lze obvykle pokračovat i s jiným počtem procesů.

Výjimkou je navázání do stejného XDMF/HDF5 výstupu s celou historií prostorových polí. Při restartu se tato dvojice souborů obnoví ze snapshotu a otevře se v režimu append; kvůli paralelnímu HDF5 je proto prakticky nejbezpečnější zachovat stejný počet MPI procesů jako v původním běhu. Pokud počet procesů změnit potřebujete, pokračujte z checkpointu výpočtového stavu, ale založte

nový XDMF výstup (nebo XDMF při pokračování vypněte). Stejný počet procesů zachovejte také při problémech se čtením checkpointu, při lokálních nesdílených adresářích na uzlech nebo při nejisté kompatibilitě prostředí.

Pokud změníte geometrii, síť, materiály nebo podstatné nastavení simulace, checkpoint adresář **smažte**. Jinak se nový běh může pokusit navázat na stav, který už neodpovídá aktuálním parametrům.

9 Meteorologie a měření

PVGIS a cache

Meteorologická data se v simulacích používají hlavně jako časové vstupy termů: typicky teplota okolí, dostupný výkon FVE nebo příznaky topné sezóny. Praktický postup je vždy podobný: data načíst, zkontrolovat časovou osu a vybraný sloupec převést na funkci $f(t)$, kterou potom předáte do `solve_unsteady` například jako `T_amb_t`.

Modul `heat_battery/data/meteodata.py` k tomu poskytuje `CachedMeteoDataLoader`. Ten umí stáhnout hodinová data přes PVGIS a uložit je do cache, aby se stejný dotaz při opakovaném běhu nestahoval znovu.

- Cache adresář se nastavuje argumentem `cache_dir`; pro stejné argumenty volání se použije stejný hash a data se načtou z lokální cache.
- V paralelním běhu data stahuje pouze rank 0 a výsledek se rozešle ostatním procesům, takže každý MPI rank neposílá vlastní REST dotaz.
- Výsledný DataFrame obsahuje sloupce pro teplotu okolí, dostupný výkon PV, čas a pomocné příznaky pro topnou sezónu a pauzy.
- Předdefinované lokality jsou ve slovníku `locations` (např. 'Brno-FME'). Vlastní lokalitu lze doplnit stejným formátem (`latitude, longitude, altitude`).

Převod sloupce na funkci $f(t)$

```
from heat_battery.data import meteodata
loader = meteodata.CachedMeteoDataLoader('weather')
df = loader.fetch_hourly(meteodata.locations['Brno-FME'])[0]
T_amb_t = meteodata.to_function(df, 'temp_air',
                                offset_date='2007-06-01 00:00:00')
```

`offset_date` určuje, kterému datu v datech odpovídá simulovaný čas $t = 0$. Funkce `to_function` vrací rychlou Python funkci $f(t)$ s lineární interpolací (volitelně i bez interpolace). Takto připravenou funkci lze předat přímo do modelu:

```
sim.solve_unsteady(
    T_amb_t=T_amb_t,
    # dalsi parametry behu...
)
```

10 Parametrické studie (ParameterGrid)

Parametrická studie je vhodná tehdy, když chcete porovnat více variant geometrie, materiálů, okrajových podmínek nebo řídicích parametrů. Místo ručního spouštění jednotlivých skriptů popíšete prostor variant pomocí `ParameterGrid` a z něj se vygenerují jednotlivé úlohy.

Základní stavební prvky jsou v modulu `heat_battery/simulations/sweep.py`:

Třída / pojem	Popis
<code>ParameterList</code>	Jedna osa studie – seznam hodnot jednoho parametru.
<code>ParameterGrid</code>	Slovník parametrů pro jednu úlohu nebo celou mřížku. Hodnoty mohou být konstanty, <code>ParameterList</code> , vnořený <code>ParameterGrid</code> nebo <code>ParameterEvaluation</code> .
<code>ParameterEvaluation</code>	Odložený výraz vyhodnocený až nad konkrétní sadou parametrů. Hodí se pro veličiny odvozené z jiných parametrů (proměnná <code>SELF</code>).
<code>NoNumericalEffect</code>	Parametr bez vlivu na numerický výsledek, například úroveň výpisu. Nepřispívá do hashe , takže jeho změna nezneplatní již spočtená data.

Při praktickém použití bývá vhodné oddělit parametry geometrie od parametrů simulace. Parametry geometrie určují síť; parametry simulace určují spuštění `solve_steady` nebo `solve_unsteady`. Generátor kombinací nastavuje `PRIORITY` tak, aby se nejdříve počítaly body s větším rozptylem v prostoru parametrů a teprve potom jemnější varianty. Podrobnosti jsou v docstringu `ParameterGrid.kde_indexes`.

Každá kombinace dostane `SIGNATURE`: hash SHA256 nad parametrickou sadou a kontextem řešící třídy. Stejný podpis znamená stejnou úlohu. Díky tomu lze poznat duplicity a zbytečně nevkládat nebo nepřepočítávat variantu, která už v projektu existuje.

11 PostgreSQL projekt, fronta úloh a worker

Základní tok

PostgreSQL vrstva slouží pro rozsáhlejší studie, kde je potřeba úlohy ukládat, sdílet mezi více výpočetními uzly a průběžně sledovat jejich stav. Typický tok je: připravit parametrickou mřížku, vygenerovat z ní úlohy, uložit je do projektu a potom spustit jeden nebo více workerů, kteří si úlohy berou z fronty.

1. Definuje se `ParameterGrid`, typicky zvlášť pro geometrii (`mesh_p`) a pro parametry simulace (`sim_p`). Příklad je v `examples/Example_05/run_example.py`.
2. Generátor úloh vytvoří iterátor objektů `Job`:

```
new_jobs_generator(  
    sim_class, mesh_builder,  
    runner='solve_unsteady',  
    p_grid=p_grid,  
    group_name=group_name,  
    group_priority=group_priority,  
)
```

Řetězec `runner` je název metody na instanci simulace. V ukázce se spouští `solve_unsteady`; pojmenované argumenty se doplň z `p_grid`.

3. Projekt úlohy uloží do databáze:

```
project = Project('nazev', if_exists='override')  
project.add_jobs(jobs_gen)
```

Stav úloh a výběr z fronty

Každá úloha má v databázi svůj stav. Worker si nevybírám libovolný řádek, ale volá metodu `Project.get_next_scheduled_job`. Do výběru vstupují úlohy ve stavech:

- SCHEDULED,
- FAILED* (libovolný řetězec začínající FAILED),
- INTERRUPTED,

Výběr se řadí podle `ORDER BY priority ASC LIMIT 1`, takže nejnižší číslo priority jde na řadu první. To umožňuje spouštět nejdříve hrubší nebo důležitější body studie a až potom jemnější varianty.

Worker

Worker je proces, který v cyklu bere úlohy z databáze a spouští `job.run()`. V tomto repozitáři jej představuje skript `worker_test.py`. Spouští se přes MPI, protože samotná simulace může být paralelní:

```
mpirun -n 4 python3 worker_test.py --num-jobs 5 --project project_example_05
```

Argument	Smysl
<code>--num-jobs, -n</code>	Maximum úloh zpracovaných jedním spuštěním workeru (výchozí 1).
<code>--project, -p</code>	Název projektu v databázi, ze kterého se mají úlohy vybírat (výchozí <code>project_example_05</code>).

Pokud fronta nevrátí žádnou úlohu, worker se ukončí. Při dlouhodobém provozu je proto vhodné spouštět ho pod dohledem `systemd`, `cronu` nebo jiného supervisoru, který ho podle potřeby znovu spustí.

Samostatný **dohledový skript** (např. `examples/Example_05/run_example.py`) obvykle úlohy do projektu přidává a průběžně kontroluje jejich stav. Podle potřeby volá `mark_interrupted_jobs()`, aby vrátil přerušené úlohy zpět do fronty.

V budoucí verzi by se tato údržba fronty měla přesunout přímo do PostgreSQL vrstvy. Cílem je, aby systém uměl podle stavů uložených v databázi automaticky poznat přerušené nebo nedokončené úlohy a vrátit je do plánování bez zvláštního dohledového skriptu. Přesný mechanismus zatím není pevně daný, ale databázový stav úloh k tomu poskytuje přirozené místo.

Export a přenos projektu

Celý PostgreSQL projekt lze uložit do jednoho souboru a přenést na jiný server. Metoda `Project.save_to_file(...)` vytvoří archiv `.tar.gz`, který obsahuje databázové tabulky projektu, binární soubory výsledků a checkpointy. Díky tomu se nepřenáší jen dokončené simulace, ale i stav rozběhnutých nebo nedokončených úloh, pokud už mají zapsaný stav v databázi a checkpoint.

```
project = Project("project_example_05")
project.save_to_file("project_example_05.tar.gz")
```

Na cílovém serveru se archiv načte do nového nebo existujícího názvu projektu:

```
project = Project("project_example_05_imported")
project.load_from_file("project_example_05.tar.gz")
```

Před exportem je nejbezpečnější zastavit workery nebo počkat na checkpoint, aby se právě zapisované výsledky nepřenesly v neúplném stavu. Po importu lze nedokončené úlohy znovu zařadit do fronty stejným dohledovým mechanismem jako při běžném přerušení výpočtu.

12 Vizualizace

Výstupy ze simulací mají dvě různé role. Pro kontrolu prostorového pole teploty se používá ParaView a XDMF soubory. Pro průběžné sledování větší parametrické studie je praktičtější webový dashboard nad PostgreSQL projektem.

Prostorová pole v ParaView

Pokud má simulace zapnutý `xdmf_file`, ukládá časovou řadu teplotního pole do dvojice `.xdmf` a `.h5` v adresáři výsledků. Tento výstup otevřete v ParaView a použijete jej hlavně pro kontrolu prostorového rozložení teploty, lokálních extrémů, chování okrajových podmínek a návaznosti výpočtu po restartu z checkpointu. XDMF je vhodné zapínat u běhů, kde opravdu potřebujete vidět celé pole; u velkých parametrických studií může zabírat výrazně víc místa než skalární sondy.

Budoucí verze webového nástroje by měla obsahovat také zobrazovač XDMF dat. Cílem je, aby bylo možné přímo v prohlížeči kontrolovat nejen časové průběhy sond, ale i tvar geometrie a prostorová pole, včetně zobrazení geometrie nebo výsledků v řezu.

Webový dashboard nad PostgreSQL projektem

Webový nástroj v `heat_battery/visualization` je Dash aplikace, která čte projekty uložené v PostgreSQL. Hodí se hlavně pro `Example_05` a podobné studie, kde běží mnoho úloh a výsledky není praktické otevírat po jednotlivých CSV souborech. Příklad nasazení je v `examples/Example_05/view_project.py`.

Typický postup je:

1. Připravte PostgreSQL projekt a nechte do něj zapsat úlohy i výsledky sond. Dashboard zobrazuje to, co je uloženo v databázi; bez dostupného projektu a výsledkových tabulek nemá co načíst.
2. V souboru `view_project.py` nastavte objekty `Project` podle názvů projektů v databázi. Jeden dashboard může registrovat více projektů současně.
3. Nastavte přístupové údaje v `credentials` a oprávnění v `user_permissions`. Pro lokální test stačí jednoduchý uživatel; pro sdílený server nastavte proměnnou prostředí `FLASK_SECRET_KEY`.
4. Upravte `initial_figures_templates`. Každá položka definuje výchozí graf: název, typ grafu, osu x , seznam sloupců na ose y a případné nastavení os. Položka `y_names` může obsahovat přesný název sloupce i regulární výraz, například `T_.*`.
5. Podle potřeby doplňte `variable_descriptions`. Tyto popisy se používají jako kontextová nápověda k jednotlivým veličinám v grafech.

python3 `examples/Example_05/view_project.py`

V ukázkovém souboru se server spouští metodou `debug_mode(host=..., port=8050)`, což je vhodné hlavně pro vývoj a rychlou lokální kontrolu. Pro běžné spuštění lze použít `start_app(host=..., port=8050)`. Po `build_app()` je k dispozici také Flask server, který lze předat WSGI serveru. Při lokálním použití obvykle stačí `127.0.0.1`; při spuštění na výpočetním serveru musí být zvolený host a port dostupný z počítače, ze kterého dashboard otevíráte v prohlížeči.

Práce v aplikaci

Dashboard má pro každý registrovaný projekt několik pohledů. **Jobs table** slouží ke kontrole fronty: vidíte stav úloh, priority, podpisy parametrických kombinací a případné chyby. Tento pohled je první místo, kde se vyplatí hledat, pokud se výsledky v grafech neobjevují nebo některé úlohy zůstávají ve stavu `FAILED`, `INTERRUPTED` nebo `RUNNING`.

Result viewer zobrazuje časové průběhy sond pro vybranou úlohu. Při výběru úlohy se pracuje se signaturou dané parametrické kombinace. Tlačítko `Refresh Data` znovu načte data z databáze; u běžící úlohy se data mohou průběžně obnovovat automaticky. Přes `Add graph` lze přidat další časový graf a přes `Data Transforms` připravit odvozené veličiny pro porovnání.

Result comparator slouží ke srovnání více úloh mezi sebou. Používejte jej hlavně tehdy, když chcete rychle posoudit vliv parametrů na teploty, energie, toky tepla nebo počty iterací řešiče. Pro dlouhé časové řady je vhodné držet počet zobrazených veličin rozumný; grafy jsou interaktivní, ale přenos a vykreslení velkého množství stop může být pomalé.

Webový dashboard nenahrazuje checkpointy ani XDMF výstupy. Čte hlavně stav úloh a skalární výsledky sond uložené v PostgreSQL. Pro detailní kontrolu teplotního pole používejte ParaView; pro rychlou kontrolu průběhu studie, porovnání variant a sdílení výsledků používejte dashboard.

13 Přehled příkladů (examples/)

Adresář `examples/` je nejlepší místo, kde začít při úpravě vlastního modelu. Jednotlivé příklady ukazují různé úrovně složitosti: od jednoduchého válcového úložiště přes měřicí experiment až po parametrickou studii s databází, workerem a webovým dashboardem.

Example_01 – základní válcové úložiště

Tento příklad je nejjednodušší šablona pro vlastní model. Geometrie obsahuje válcové úložiště s pojmenovaným objemem topné patrony a s vnější hranicí pro ochlazování do okolí.

- **Termy:** `UniformHeatSource` přidává objemový zdroj tepla do oblasti `"heated cartridge"`. `AmbientCooling` přidává konvektivní okrajovou podmínku na `"outer_surface"`.
- **Sondy:** `T` odečítá teploty v bodech definovaných v metadatech geometrie. `heat` integruje uloženou energii přes materiálové části domény. Prakticky tak vidíte lokální teploty i celkový stav nabití úložiště.
- **Jak číst kód:** začněte v `geometry.py`, kde se staví Gmsh model, fyzikální skupiny, materiály, okrajové části a body sond. V `model.py` je třída `Experiment_v1`; její metody připojí Termy, zaregistrují sondy a nastaví vstupy pro stacionární i nestacionární běh. `run_example.py` potom připraví geometrii, založí simulaci a uloží výsledky.

Základní vazba mezi geometrií a rovnicí je v `model.py` velmi krátká:

```
def define_form_subdomain_terms(self):
    self.set_source_term(
        terms.UniformHeatSource(self),
        "heated cartridge",
    )
    self.set_bc_term(
        terms.AmbientCooling(self),
        "outer_surface",
    )
```

Názvy "heated cartridge" a "outer_surface" nejsou libovolné řetězce. Musí odpovídat metadatům geometrie. Díky tomu je fyzika modelu oddělená od konstrukce sítě: geometrie říká, *kde* je patrona a vnější hranice, zatímco Term říká, *jaký příspěvek* se na daném místě vloží do slabé formulace.

Stejný princip platí pro sondy:

```
coords = list(self.geo_meta["points"]["T"].values())
sampler = FunctionSampler(coords, self.domain)

@probes.register_probe("T", unit="degC")
def T_probe():
    return sampler.eval(self.T)
```

Body sond jsou uloženy v metadatech geometrie, ne natvrdo v metodě sond. Model tak nemusíte měnit při každé změně polohy teplotních čidel; stačí upravit `geometry.py`. Objekt `FunctionSampler` si připraví vyhodnocení MKP funkce v daných bodech a sonda už jen vrací aktuální hodnoty teplotního pole.

Example_02 – stejný model s importovanou geometrií

Example_02 používá stejnou simulační strukturu jako Example_01, ale ukazuje práci s vlastní geometrií importovanou z CAD souboru. Je vhodný jako výchozí bod, pokud už máte síť nebo tvar připravený mimo parametrický generátor.

- **Termy:** zůstává `UniformHeatSource` na topné oblasti a `AmbientCooling` na vnější hranici. Důležité je, aby importovaná geometrie měla správně pojmenované fyzikální skupiny.
- **Sondy:** stejně jako v Example_01 se zapisují bodové teploty `T` a integrované teplo `heat`. Tento příklad tedy ověřuje hlavně to, že se vlastní geometrie správně napojila na stejný model.
- **Jak číst kód:** `model.py` je téměř stejný jako v Example_01, proto se soustředte hlavně na `geometry.py`. Tam je vidět import STEP geometrie, pojmenování objemů a hranic a doplnění metadat pro materiály a sondy. `run_example.py` ukazuje, že po načtení geometrie už se s modelem pracuje stejně jako s parametricky vytvořenou sítí.

Nejdůležitější kód je zde v `geometry.py`. Místo ručního skládání CAD modelu se načte STEP soubor a k jeho tagům se doplní význam:

```
path = f"{stp_dir}/Experiment_v1.stp"
mats = {
    "sand": (materials.SandTheory, [4]),
    "heated cartridge": (materials.Cartridge_heated, [7]),
    "walls": (materials.Steel04, [1]),
}
bcs = {
    "outer_surface": [12, 20, 21, 22, 23, 24, 25, 26],
}

build_geometry_from_stepfile(
    path=path,
    dir=g_dir,
    name="inventor_v1",
    points=points,
    mats=mats,
    bcs=bcs,
    extract_axisymetry=True,
)
```

Tento úsek ukazuje, proč je u importované geometrie zásadní správné pojmenování. Tagy z CAD/Gmsh se převádějí na materiálové oblasti a okrajové části, které pak použije `model.py`. Poslední volba v ukázce říká, že se z importovaného tvaru připraví osově symetrická výpočetní úloha.

Example_03 – transient hot-wire experiment

Tento příklad modeluje měření typu transient hot wire. Dvě vodičové oblasti se ohřívají elektrickým proudem a z teplotní odezvy a odporu vodičů se následně vyhodnocují veličiny související s tepelnou vodivostí.

- **Termy:** odporový Term pro dvojici vodičů počítá elektrický ohřev v dlouhém a krátkém vodiči podle proudu a teplotně závislého odporu. `AmbientCooling` uzavírá model na vnější hranici.
- **Sondy:** kromě bodových teplot `T` a celkového tepla `heat` se zapisují odpory dlouhého a krátkého vodiče, jejich rozdíly, proud, napětí, elektrické výkony a teploty odvozené z odporu. Sondy zde neslouží jen k monitoringu simulace, ale přímo k rekonstrukci měřené odezvy experimentu.
- **Jak číst kód:** v `geometry.py` jsou důležitá hlavně metadata vodičů, například jejich délky a průměry. `model.py` obsahuje třídu `THW_twowire`; metoda `run_experiment` nastaví proud tak, aby odpovídal zadanému výkonu, a potom spustí nestacionární výpočet. Metoda `calculate_lmbd` je následné zpracování CSV výsledků, ne součást samotné slabé formulace.

Připojení Termů ukazuje, že jeden objekt Termu může obsluhovat více subdomén:

```
wire_heating = terms.SerialWiresResistiveHeating(self)
self.set_source_term(wire_heating, "long_wire")
self.set_source_term(wire_heating, "short_wire")
self.set_bc_term(terms.AmbientCooling(self), "outer_surface")
```

Je to záměrné: dlouhý i krátký vodič jsou součástí jednoho elektrického zapojení, proto sdílejí stejný proud a stejnou logiku odporového ohřevu. Rozlišení, ve které části domény se příspěvek integruje, se provede až názvem subdomény.

V metodě `run_experiment` se nastavuje proud podle aktuálního odporu:

```
def constQ_update(t):
    R = self.unsteady_probes.get_value("R")
    return np.sqrt(P/R*1000)

self.get_source_term("long_wire").set_update(
    "current",
    constQ_update,
)
```

Tím se místo předepsaného proudu realizuje přibližně konstantní elektrický výkon. Aktuální odpor `R` je sonda, takže řízení zdroje používá stejný mechanismus jako běžné výstupy simulace. To je typický příklad vazby mezi Termem a sondami.

Example_04 – pasivní úložiště s PID řízením patrony

Example_04 rozšiřuje základní úložiště o řízení výkonu topné patrony. Model má vnější povrch a samostatnou vnitřní okrajovou část reprezentující teplosměnný povrch pro odběr tepla.

- **Termy:** `PIDControlledHeatSource` řídí výkon v oblasti "heated cartridge" podle teplotní sondy. Dva Termy `AmbientCooling` popisují výměnu tepla na vnějším povrchu a na povrchu "membrane_surface".

- **Sondy:** zapisují se bodové teploty T , uložené teplo $heat$, tepelné ztráty přes vnější povrch a membránový povrch, okamžitý výkon patrony $power$ a průměrné teploty vybraných povrchů T_{s_avg} , T_{m_avg} , T_{c_avg} . Sonda `PID` ukazuje aktuální stav regulátoru, takže lze kontrolovat, proč byl zvolený právě daný výkon.
- **Jak číst kód:** `geometry.py` připravuje importovanou geometrii, plochu patrony pro průměrnou teplotu a okrajové části pro ztráty. V `model.py` je klíčová metoda `solve_unsteady`: nejprve nastaví `PID` regulátor, jeho referenci, limity výkonu a sonda T_{c_avg} , podle které se regulátor rozhoduje. Teprve potom se spustí obecný nestacionární řešič.

V tomto příkladu je podstatné nastavení `PID` regulátoru před spuštěním výpočtu:

```
obj = self.get_source_term("heated cartridge")
obj.set_pid(*pid)
obj.set_output_limits(pid_power_lims)
obj.set_ctrl_interval(pid_ctrl_interval)
obj.set_reference(T_pid_input_control)
obj.set_probe(lambda t: self.unsteady_probes.get_value("Tc_avg"))
```

Term tedy nedostává výkon přímo. Dostane referenční teplotu, limity výkonu a funkci, ze které si v každém kroku přečte regulovanou veličinu. Zde je to sonda T_{c_avg} , tedy průměrná teplota povrchu patrony. Takové uspořádání je praktičtější než pevně zadrátovat konkrétní sondu do Termu: stejný regulátor lze později napojit na jinou veličinu.

Example_05 – parametrická studie pasivního úložiště

Example_05 je hlavní ukázka většího pracovního postupu. Modeluje pískové úložiště nabíjené z FVE, odběr tepla do haly C3, bivalentní zdroj a rozsáhlou parametrickou studii nad geometrií i provozními parametry.

- **Termy:** `TemperatureLimitedUniformHeatSource` převádí dostupný výkon FVE na teplo v patronách, ale hlídá teplotní limit a může výkon omezit. Vlastní Term `HallC3` reprezentuje budovu: počítá teplotu místností, ztrátu do okolí, odběr tepla z povrchu úložiště a z THP povrchů, bivalentní výkon, integrály energií a adaptaci časového kroku podle změny pokojové teploty.
- **Sondy:** kromě teplotních sond T a energie $H_{storage}$ se zapisují výkony Q_{pv} , Q_c , Q_s , Q_m , Q_{s2r_total} , $Q_{bivalent}$ a Q_{amb} . Energetické sondy H_* sledují, kolik energie prošlo FVE, patronami, úložištěm, místností a bivalentním zdrojem. Teplotní průměry T_{avg_sand} , T_{avg_m} , T_{avg_s} a T_{avg_room} popisují stav úložiště i místností. Sondy Eff_* převádějí energetickou bilanci na účinnosti a podíly, které se dobře porovnávají mezi variantami.
- **Jak číst kód:** `geometry_v7.py` generuje variantní geometrii úložiště, patron a THP povrchů. V `model.py` čtete nejdříve Term `HallC3`, protože popisuje vazbu na budovu, pokojovou teplotu, bivalentní zdroj, integrály energií a adaptaci časového kroku. Třída `C3_passive` potom tento Term připojí na povrchy úložiště. Metoda `solve_unsteady` načte meteorologická data, vytvoří časové funkce pro venkovní teplotu a FVE výkon a nastaví limity patron i parametry budovy.

Vlastní Term `HallC3` je zjednodušený model budovy. Definuje veličiny, které se potom používají jak v okrajové podmínce, tak v sondách:

```

self.new_constant("T_amb", self.T_room_ref)
self.new_constant("Q_amb", 0.0)
self.set_update("Q_amb", self.Q_amb_update, prevent_override=True)

self.new_constant("Q_bivalent", 0.0)
self.set_update("Q_bivalent", self.Q_bivalent_update,
                prevent_override=True)
self.new_integral("Q_bivalent_cumulative", 0.0, "Q_bivalent")

```

Konstanty vstupují do slabé formulace nebo do výpočtu dalších veličin. Funkce `set_update` říká, jak se mají aktualizovat v čase. Integrál z `Q_bivalent` pak dává kumulovanou energii dodanou záložním zdrojem. Proto se ve výsledcích objevují okamžité výkony $\dot{Q}_*$ i energie H_* .

Nestacionární běh propojí meteorologii, FVE a Termy:

```

meteo_loader = meteodata.CachedMeteoDataLoader("weather")
meteo_data = meteo_loader.fetch_hourly(location)[0]

T_amb_t = meteodata.to_function(meteo_data, "temp_air",
                                offset_date=datetime_start)
P_pv_t = meteodata.to_function(meteo_data, "P",
                                offset_date=datetime_start)

obj_cartridge = self.get_source_term("Cartridge")
obj_cartridge.set_update("Q_in", lambda t: pv_peak/1000*P_pv_t(t))
obj_cartridge.set_T_limit(Tc_limit)

```

Tady je vidět, proč se meteorologická data převádějí na funkce času. Řešič pracuje s časem t , ne s tabulkou hodinových hodnot. Term topné patrony proto v každém kroku dostane aktuální dostupný výkon FVE a zároveň hlídá teplotní limit patrony.

Praktické soubory v Example_05:

- `run_example.py` – naplňuje frontu úloh do PostgreSQL a hlídá běžící úlohy. Při čtení se zaměřte na definici `ParameterGrid`, generování úloh a práci s objektem `Project`.
- `run_example_size5.py` – přímý lokální běh jednoho případu bez databáze, s checkpointy a XDMF výstupem. Je vhodný pro pochopení jednoho konkrétního běhu bez databázové vrstvy.
- `view_project.py` – webový dashboard pro sledování fronty, sond a porovnání výsledků. Zde se nastavují projekty, výchozí grafy, popisy veličin a přístupová práva.

Lokální běh jednoho případu obchází databázi a je vhodný pro ladění:

```

sim = C3_passive(
    geometry_dir=g_dir,
    model_name=model_name,
)
sim.solve_unsteady(
    result_dir=r_dir,
    xdmf_file="unsteady.xdmf",
    checkpoint_dir=checkpoint_dir,
)

```

Tento vzor je užitečný, když chcete nejdříve ověřit jednu konkrétní variantu geometrie a nastavení. Teprve po takové kontrole dává smysl stejný model zapojit do `ParameterGrid` a PostgreSQL fronty.

Example_06 – aktivní úložiště s řízeným odběrem

Example_06 používá podobnou logiku jako Example_05, ale v jednodušší geometrii s jedním membránovým povrchem. Je vhodný pro pochopení vazby mezi úložištěm, řízeným odběrem tepla a zjednodušeným modelem vytápěné místnosti.

- **Termy:** `TemperatureLimitedUniformHeatSource` znovu popisuje nabíjení patron z FVE s teplotním omezením. Term `HallC3` je připojený na `"Outer_surface"` a `"Membrane_surface"`; podle teploty místnosti, venkovní teploty a dostupného odběru nastavuje tok tepla z úložiště a případný bivalentní výkon.
- **Sondy:** sledují stejnou bilanci jako Example_05, ale pro jednodušší členění povrchů: teploty `T`, `T_avg_sand`, `T_avg_m`, `T_avg_s`, `T_avg_room`, výkony `Q_*`, energie `H_*`, příznak topné sezóny `Heating_on` a účinnosti `Eff_*`. Díky tomu lze rychle ověřit, zda řízení skutečně pokrývá poptávku po teple a kolik energie z FVE se využije.
- **Jak číst kód:** `geometry_v4.py` ukazuje zjednodušený geometrický model se symetrií a jedním agregovaným membránovým povrchem. V `model.py` je struktura podobná Example_05, ale bez segmentace THP povrchů; proto se lépe čte při studiu samotné logiky řízení. `run_example.py` je vhodný pro kontrolu restartů a porovnání běhů uložených do různých CSV souborů.

Rozdíl proti Example_05 je dobře vidět v připojení okrajových Termů:

```
hr_obj = HallC3(self)
self.set_bc_term(hr_obj, "Outer_surface")
self.set_bc_term(hr_obj, "Membrane_surface")
```

Místo mnoha segmentů THP povrchů je zde jeden agregovaný `"Membrane_surface"`. Kód je tím kratší a čitelnější, ale stále ukazuje stejnou myšlenku: Term budovy je okrajová podmínka, která z teploty povrchu úložiště počítá tok tepla do místnosti. Proto je Example_06 vhodný, pokud chcete nejdříve pochopit řízení a checkpointy bez celé geometrické složitosti Example_05.

14 Optimalizace a korektnost úlohy

Pozn. terminologie: V kódu se používá anglické `posedness` (`get_optimization_problem_posedness`); v češtině jde o **korektnost (podmíněnost) úlohy**.

SteadyStateComparer

`heat_battery/optimization/optimization.py` porovnává sadu **stacionárních scénářů** (inputs jsou kwargs do `solve_steady`) s naměřenými **výstupy v sondách** (outputs):

- `generate_loss_gradient_for_material` – adjoint gradient účelové funkce vůči koeficientům vodivosti k (po `batch_size` scénářích).
- `generate_solution_jacobian` – citlivost $\partial u / \partial k$ (na sondu, nebo s `full_domain=True` celá doména).
- `get_optimization_problem_posedness` – **SVD** Jakobiánu. Výsledkem je `JacobianRankAssessment` se singulárními hodnotami, tvarem matice, efektivním rankem a použitým limitem σ .

Jak číst výstup

- Singulární hodnoty blízko nuly označují směry v prostoru k , které **téměř nemění predikci** modelu na sondě. To obvykle znamená slabě navržený experiment nebo redundantní data.
- Platí $\text{effective_rank}_* \leq \min(n_k, n_o)$ (n_k parametry, n_o počet měření $\times$ scénáře). Pokud je efektivní rank výrazně menší než n_k , úloha je **špatně podmíněná** a bez regularizace nebo bohatší sady experimentů (jiné výkony, teploty okolí...) se řešení nebude dát spolehlivě určit.
- Jde o **deterministickou** citlivost modelu v daném bodě k – nenahrazuje statistiku šumu měření.

Optimalizátory

Modul `heat_battery/optimization/optimizers.py` sdružuje gradientní, Newtonovské i náhodné vyhledávací metody. Patří sem například ADAM, Newton, NewtonLineSearch, GaussNewtonDescent, GradientSearch, QuasiNewtonBFGS a RandomDirectionSearch. Všechny používají společné rozhraní `optimise(...)` s tolerancemi a callbackem.

15 Ověření funkčnosti softwaru

Funkčnost softwaru se průběžně ověřuje na sadě příkladů v adresáři `examples/`. Během vývoje sloužily jako regresní testy modelů, geometrie, sond, checkpointů i ko-simulace; zároveň tvoří praktický tutoriál. Každý příklad má vlastní `README.md` s popisem fyzikální úlohy, použitých materiálů, okrajových podmínek, sledovaných veličin a způsobu spuštění.

Automatické testy v adresáři `test/`

Kromě ruční kontroly příkladů obsahuje repozitář i sadu automatických testů postavených na standardním Python modulu `unittest`. Spouštějí se jen soubory odpovídající vzoru `test_*.py`; pomocné nebo experimentální testy pojmenované `no_test_*.py` do běžného testovacího běhu nepatří.

```
python3 -m unittest discover -v -s ./test -p "test_*.py"
```

Testy nepokrývají pouze import balíčku, ale i numericky důležité části kódu:

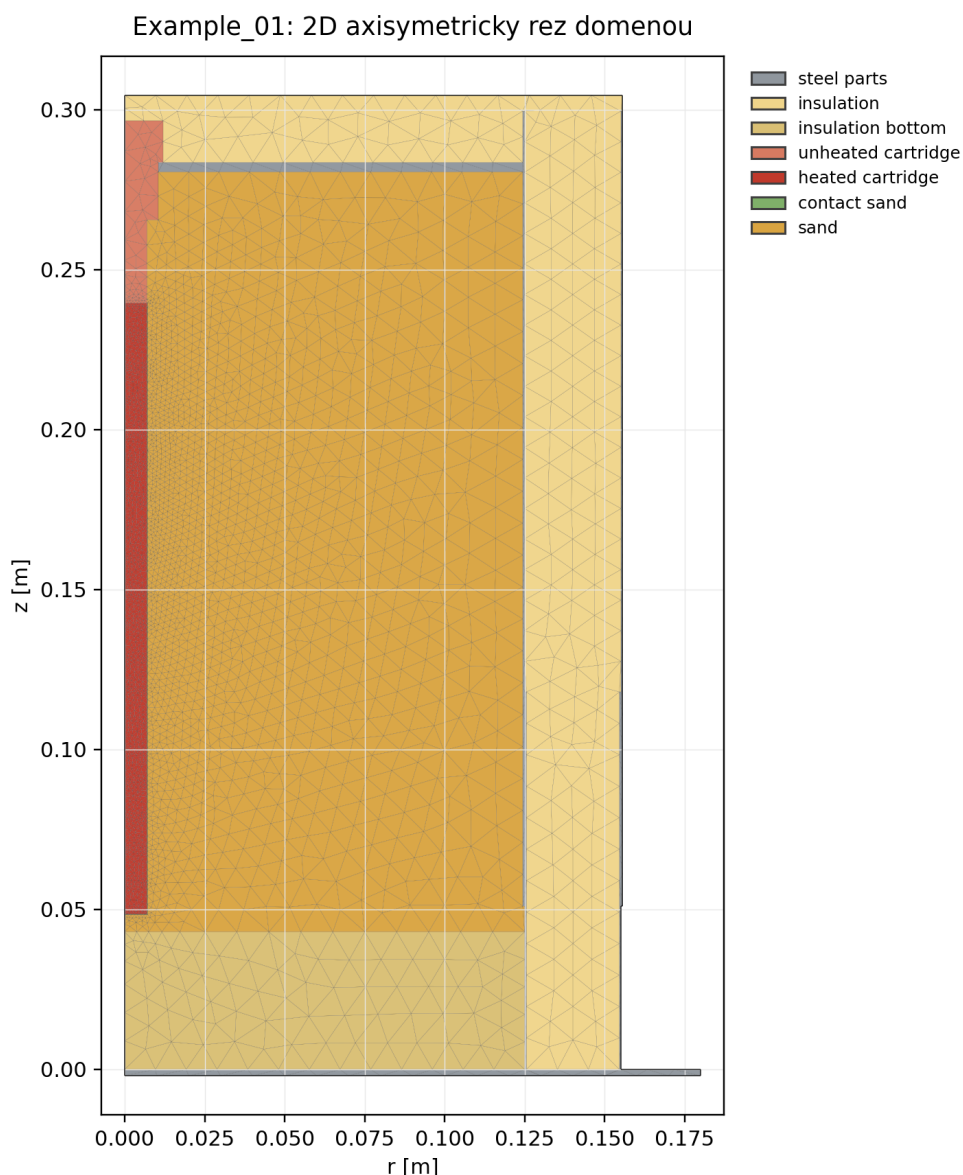
- **Základní běh simulace** – `test_simulation_solve.py` spouští stacionární i krátký nestacionární výpočet `Example_01` a porovnává vybrané hodnoty sond s referenčními čísly.
- **Příklady** – `test_examples.py` postupně spouští `Example_01` až `Example_04`. Tím se kontroluje, že geometrii, modelovou třídu, termy a výstupy lze sestavit v běžném uživatelském režimu.
- **Geometrie a materiály** – `test_experiment_geometry.py` a `test_materials.py` ověřují Gmsh/STEP workflow, mapování fyzikálních skupin a API pro materiálové vlastnosti.
- **Parametrické studie a databáze** – `test_sweep.py` kontroluje chování `ParameterGrid`, zatímco `test_postgresql.py` ověřuje ukládání a ekvivalenci úloh v PostgreSQL projektu.
- **Optimalizace** – testy `test_derivative.py`, `test_steady_state_fitter.py` a `test_optimizer_*.py` kontrolují derivace, adjoint gradienty a chování optimalizačních metod na malém modelu `Example_01`.

Tyto testy jsou hlavně regresní pojistka pro vývojáře. Pokud některý selže, je potřeba nejdříve rozlišit, zda jde o fyzikální změnu modelu, numerickou změnu řešiče, problém prostředí (například Gmsh, DOLFINx, MPI nebo PostgreSQL), nebo jen změnu referenčních hodnot po vědomé úpravě implementace.

Přehled testovacích příkladů

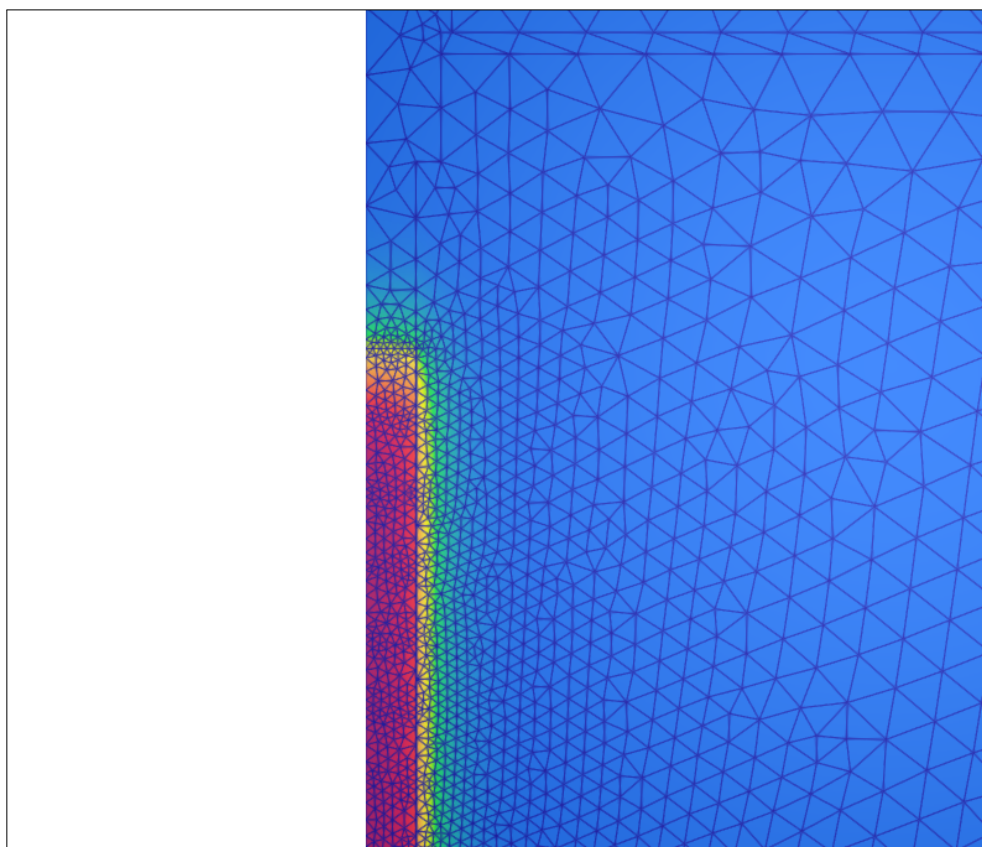
- **Example_01** – dynamický 2D-axisymetrický model malého laboratorního akumulátoru (válcová nádoba s pískem, jedna topná patrona, dvojitá ocelová stěna s minerální izolací). Geometrie je **generována procedurálně** v Gmsh, čímž lze snadno měnit rozměry a hustotu sítě. Slouží jako referenční minimální simulace (jeden zdroj, jedna konvektivní okrajová podmínka, 15 teplotních sond a sonda celkové entalpie). Součástí adresáře je také skript `trial_real_fitting_test.py`, který ukazuje napojení na reálná experimentální data: načte ustálené měření, porovná teplotní sondy se stacionárním výpočtem a pomocí `SteadyStateComparer` ladí materiálovou vodivost.

Obrázek domény ukazuje 2D-axisymetrický řez výpočtovou oblastí. Barevně jsou odlišeny materiálové podoblasti z Gmsh Physical Groups: písek, topná a netopná část patrony, kontaktní vrstva, izolace a ocelové části nádoby.



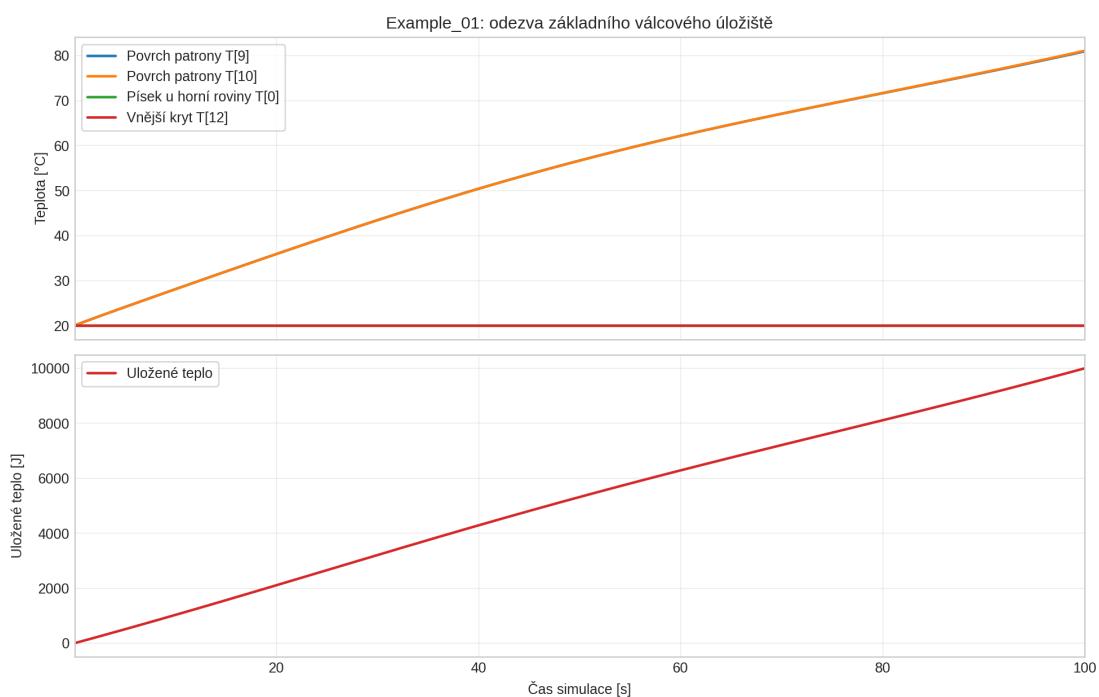
Example_01: výpočtová doména a materiálové podoblasti.

Stejný XDMF výstup lze otevřít v ParaView a zkontrolovat prostorové rozložení teploty v doméně. V řezu je dobře vidět lokální ohřev v okolí patrony a pomalejší šíření tepla do vzdálenějšího zásypu.



Example_01: teplotní pole z XDMF výstupu zobrazené v ParaView.

Obrázek ukazuje základní odezvu modelu při krátkém nestacionárním běhu. Horní graf porovnává vybrané teplotní sondy, spodní graf ukazuje růst celkového uloženého tepla. Je na něm vidět, že nejrychleji reaguje oblast v okolí topné patrony, zatímco vzdálenější body zůstávají během krátkého běhu téměř na počáteční teplotě.



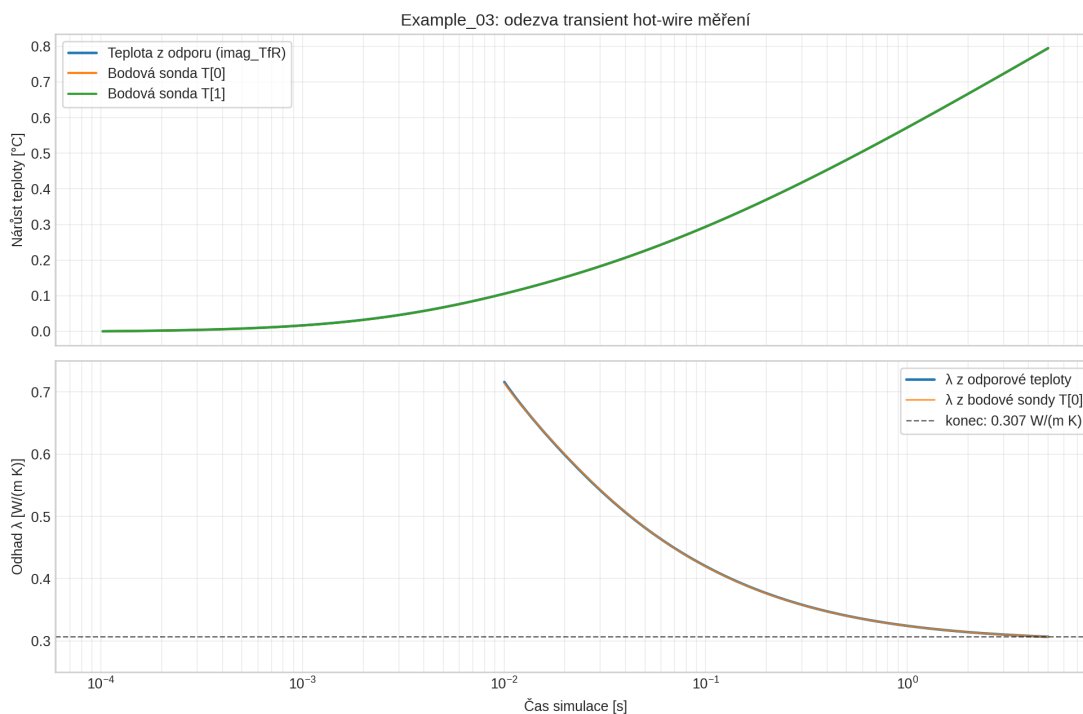
Example_01: základní teplotní odezva a nárůst uloženého tepla.

- **Example_02** – fyzikálně shodný s Example_01, ale **geometrie je importována ze STEP souboru** vytvořeného v CAD softwaru (Autodesk Inventor). Demonstruje cestu pro složitější geometrie, které není praktické skládat algoritmicky, a použití automatického *extract_axisymetry* z 3D modelu.
- **Example_03** – simulace metody *transient hot wire* (THW) pro měření tepelné vodivosti zásypu. V geometrii dvoudrátové sondy se počítá časový vývoj odporu obou tantalových vláken (proudově buzených konstantním výkonem), z něhož se přes Wheatstoneův můstek dopočítává tepelná vodivost. Příklad pokrývá nelineární zdrojový člen závislý na teplotě a post-processing CSV výstupu.

Obrázek ukazuje typický výstup z post-processingu tohoto příkladu. Horní graf zobrazuje nárůst teploty v čase. Teplota odvozená z odporu vodiče se prakticky překrývá s bodovou MKP sondou v místě vodiče; tím se ověřuje, že odporový Term, odečet teploty i následný post-processing dávají konzistentní výsledek. Spodní graf ukazuje odhad tepelné vodivosti

$$\lambda = \frac{q_l}{4\pi dT/d \ln t},$$

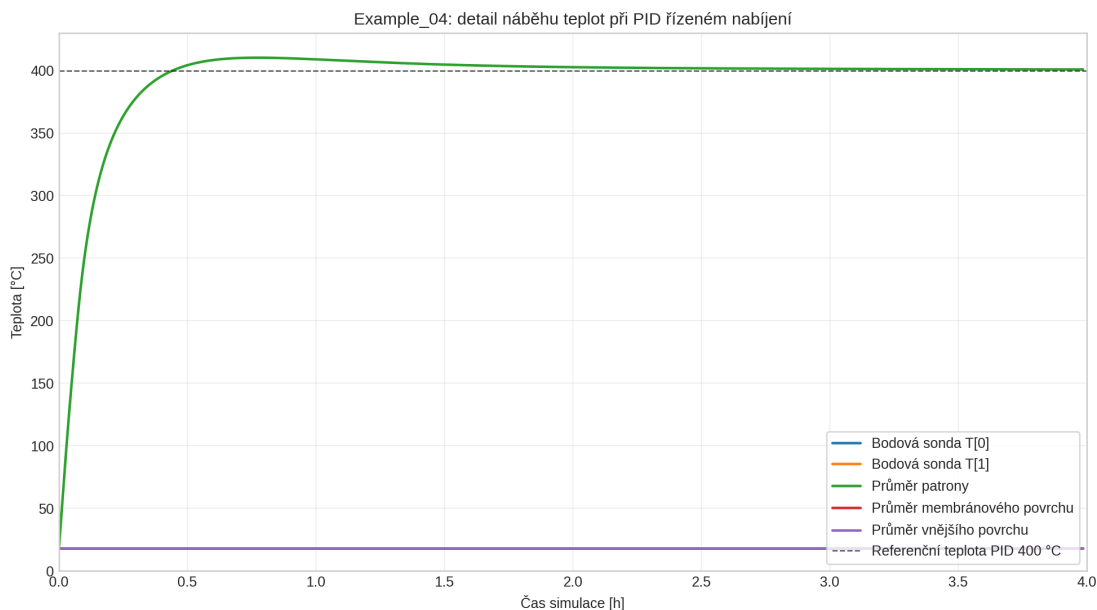
kde q_l je délkový tepelný výkon vodiče. Logaritmická osa času odpovídá způsobu, jakým se data z transient hot-wire měření obvykle vyhodnocují. Tepelná vodivost je vyčítána v okamžiku kde je průběh odporové sondy v logaritmickém měřítku lineární.



Example_03: odezva transient hot-wire měření a odhad tepelné vodivosti.

- **Example_04** – pasivní úložiště s **PID-řízenou topnou patronou**. Geometrie je importována ze STEP (Passive_storage.stp), regulátor řídí výkon patrony tak, aby průměrná teplota povrchu patrony sledovala zadanou referenci (výchozí hodnota 400 °C). Demonstruje použití termu `PIDControlled HeatSource`, který má vlastní vnější iterační smyčku (`converged / adaptation`) zajišťující implicitní vazbu mezi teplotou a akčním zásahem.

Obrázek níže ukazuje detail prvních čtyř hodin běhu. Tento rozsah je zvolený proto, aby byl dobře vidět náběh PID regulace: průměrná teplota povrchu patrony T_{c_avg} se rychle přiblíží k referenci 400 °C, zatímco teploty v tělese úložiště a na okrajových površích se v tomto krátkém čase téměř nemění.



Example_04: detail náběhu teplot při PID řízeném nabíjení topné patrony.

- **Example_05** – simulace tepelného zásobníku nabíjeného z FVE, který dodává teplo do výrobní haly (Hala C3 FSI VUT Brno). Obsahuje plnohodnotnou **ko-simulaci se zjednodušeným modelem haly** (HallC3), čerpání meteorologických dat z PVGIS (NSRDB), ekvitermní regulaci a bivalentní záložní zdroj. **Parametrická studie** (ParameterGrid) prochází hlavní rozměry úložiště a topidel na dvou letech provozu; jednotlivé kombinace se distribuují přes PostgreSQL frontu a MPI workery. Jde o nejkomplexnější příklad v repozitáři: využívá téměř všechny hlavní moduly balíčku, od geometrie a meteorologických dat přes termy, ko-simulaci, sondy a databázový projekt až po webovou vizualizaci. Jeho výsledky, včetně obrázků webového rozhraní, jsou uvedeny v následující podkapitole.
- **Example_06** – jednodušší varianta Example_05 použitá pro **validaci checkpoint mechanismu** (běh napřímo, bez databáze). Stejný model spustí třikrát: jednou souvisle, jednou s uloženým checkpointem v polovině a jednou pokračováním z toho checkpointu – výsledné sondy Q_{amb} musí být v rámci tolerance shodné, čímž se ověří úplnost serializace stavu (DoFs, konstanty termů, T_n , délka CSV souboru, režim XDMF).

V každém adresáři `examples/Example_*/` jsou kromě README také soubory `geometry*.py`, `model.py` a `run_example*.py`; dohromady tvoří kompletní postup od geometrie a sítě až po výstupy. Většinu příkladů lze spustit jedním příkazem, například:

```
mpirun -n 4 python3 -m examples.Example_xx.run_example
```

Ukázky výsledků z Example_05

Následující obrázky jsou přímo z webového rozhraní nad PostgreSQL projektem **Example_05**. Ukazují, jak se výsledky parametrické studie čtou v praxi: nejdříve se v tabulce vybere konkrétní úloha, potom se v prohlížeči výsledků zobrazí časové průběhy sond. Pro ukázkou jsou vybrány konfigurace s různou velikostí akumulátoru a různým počtem topných patron, protože právě tyto parametry výrazně ovlivňují schopnost systému přijmout výkon z FVE.

Show/hide columns			Add job		Refresh table								
Selection	Owner	Actions	GP	P	Status ↑	Progress	Remaining	Elapsed	Last update	Last checkpoint	Group	Inserted	Active node
☐ 6c5b35...		Actions ▾	0	0	COMPLETED		none	1d 7h	1d 12h ago	1d 12h ago	Example_5	5d 21h ago	UNASSIGNED
☐ 1edc66...		Actions ▾	0	1	COMPLETED		none	2d 23h	2d 21h ago	2d 22h ago	Example_5	5d 21h ago	UNASSIGNED
☐ 4fac01...		Actions ▾	0	260	INTERRUPTED		1d 15h remains	8h 57m	20h 36m ago	20h 58m ago	Example_5	5d 21h ago	UNASSIGNED
☐ 0fb55d...		Actions ▾	0	261	RUNNING		2d 0h remains	12h 6m	now	18m 26s ago	Example_5	5d 21h ago	numlab-X11DAI...
☐ 96d20f...		Actions ▾	0	483	SCHEDULED		The remaining time in seconds: 173085.2267728721s	none	5d 21h ago		Example_5	5d 21h ago	

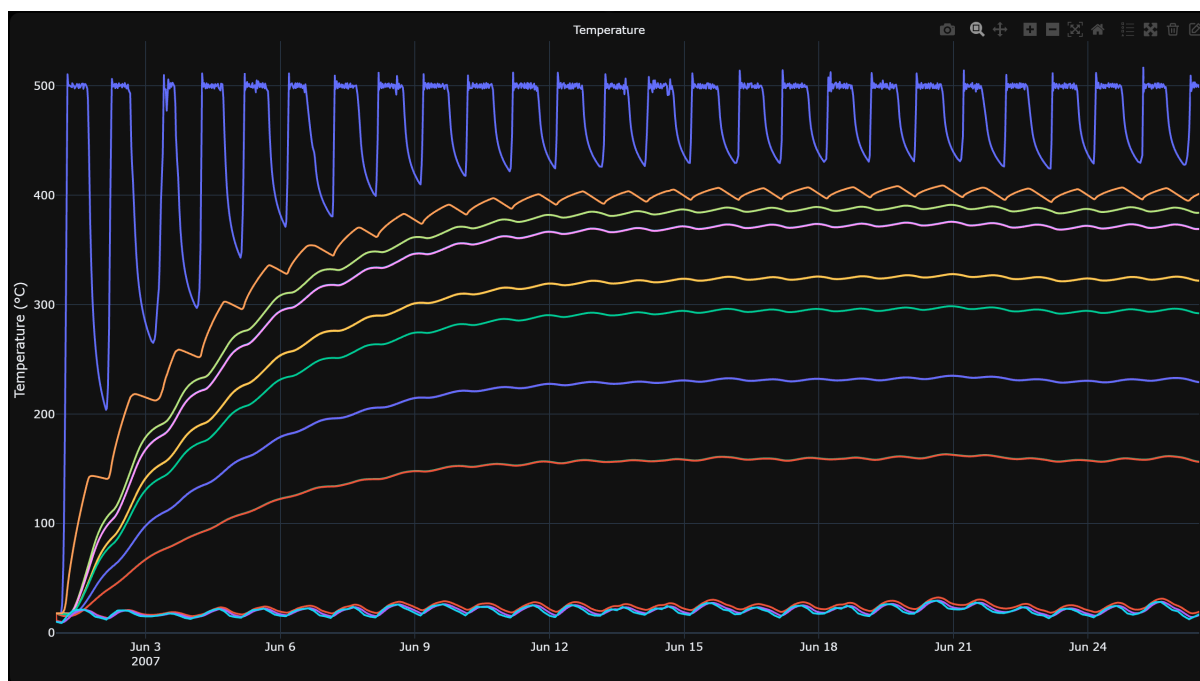
Obrázek 1: Tabulka úloh v dashboardu. Sloupce P a Status umožňují rychle najít dokončené, běžící nebo přerušené simulace; zvolenou úlohu lze otevřít v prohlížeči výsledků.

Souhrnný pohled pro malý akumulátor o průměru a výšce přibližně 1×1 m ukazuje všechny hlavní skupiny veličin najednou: teploty v akumulátoru, tepelné toky, celkovou energii, účinnosti a chování nelineárního řešiče. Takový přehled je vhodný pro první kontrolu, zda simulace došla fyzikálně smysluplně, zda nedochází k přehřívání topidel a zda solver nemá v některých obdobích výrazně vyšší počet iterací.



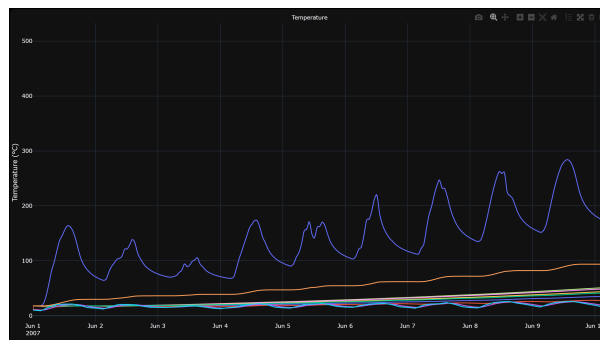
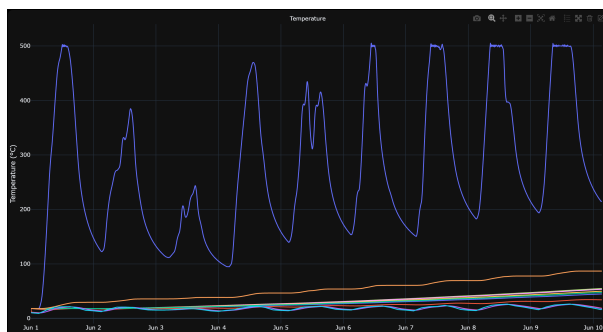
Obrázek 2: Dashboard pro malý akumulátor 1×1 m: teploty, toky, energie, účinnosti a iterace řešiče v jednom souhrnném pohledu.

Detail teplot pro malý akumulátor o průměru a výšce přibližně 1×1 m se čtyřmi patronami ukazuje rychlý ohřev topných elementů až k hornímu teplotnímu omezení. Vnitřní sondy reagují pozvolněji, protože teplo se ze zdrojů šíří do zásypu postupně. Tento případ je vhodný pro kontrolu ochrany topidel a vazby mezi dostupným výkonem z FVE, Termem topné patrony a teplotními sondami v zásobníku.



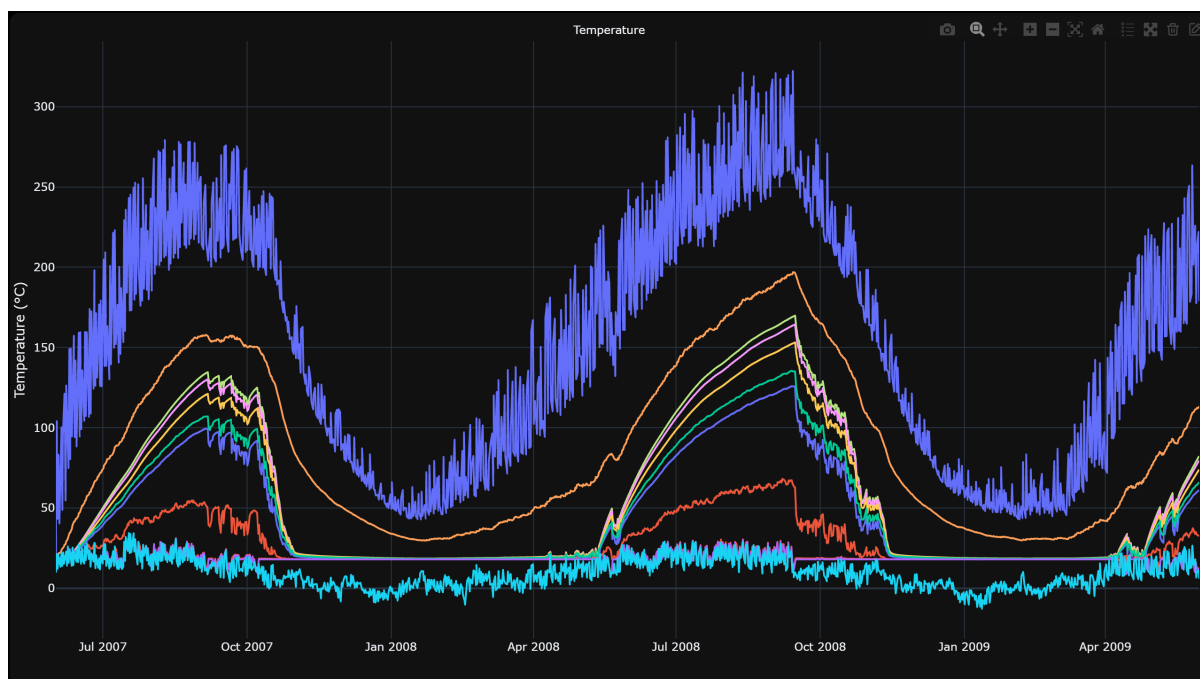
Obrázek 3: Malý akumulátor 1×1 m se čtyřmi patronami: rychlý ohřev topidel a postupné nabíjení teplotních sond uvnitř akumulátoru.

Porovnání akumulátoru o průměru a výšce přibližně 3×3 m ukazuje vliv počtu topných patron. Při čtyřech patronách se lokální teplota topidel často blíží limitu, zatímco zbytek akumulátoru se ohřívá pomalu. Po zvýšení počtu patron na deset se stejný vstupní výkon rozloží do většího objemu: teplotní špičky jsou nižší a nabíjení je rovnoměrnější. Tato dvojice obrázků dobře ukazuje, proč nestačí navrhovat jen celkovou kapacitu zásobníku; důležité je i prostorové rozložení zdrojů tepla.

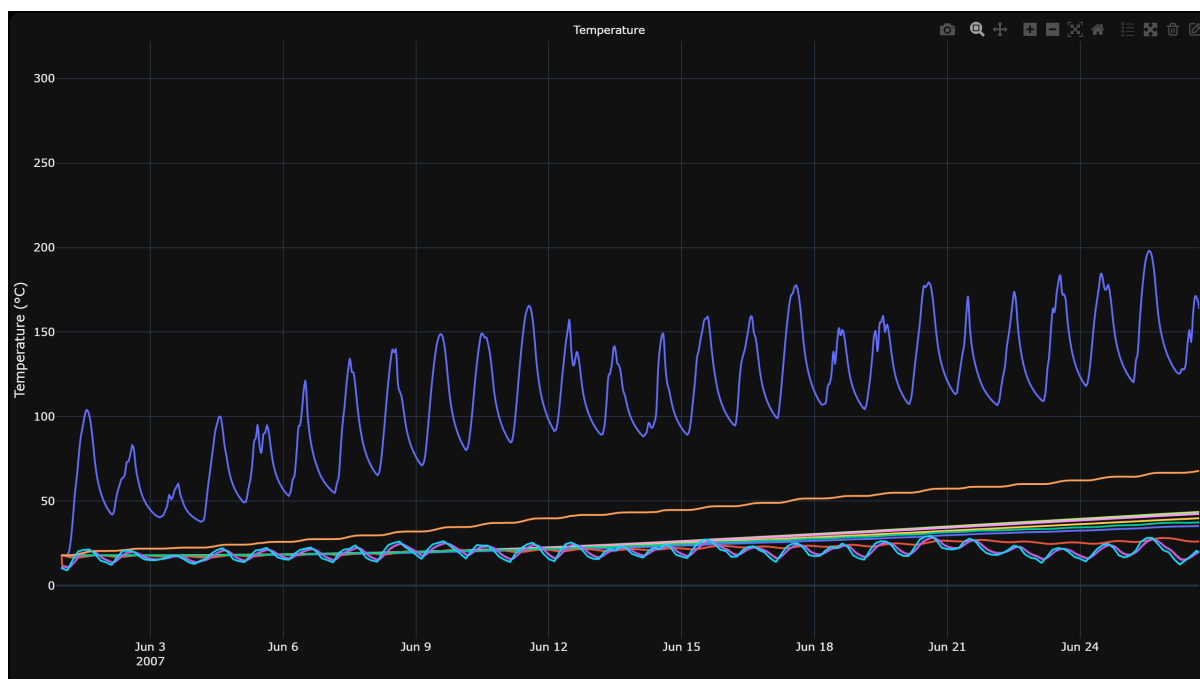


Obrázek 4: Akumulátor 3×3 m: vlevo čtyři patrony s výraznými lokálními špičkami, vpravo deset patron s rovnoměrnějším rozložením výkonu.

Velký akumulátor o průměru a výšce přibližně 5×5 m s deseti patronami se chová jinak: neukazuje jen krátkodobé denní cykly, ale také sezónní nabíjení a vybíjení. V létě roste teplota větší části zásobníku, v zimě se akumulátor postupně vybíjí do haly. Detail prvního měsíce lépe ukazuje denní pulzy od FVE a pomalé zvyšování teploty hmoty akumulátoru.



Obrázek 5: Velký akumulátor 5×5 m s deseti patronami: dvouletý průběh teplot ukazuje sezónní práci zásobníku.



Obrázek 6: Detail velkého akumulátoru 5×5 m: denní teplotní pulzy od FVE a pomalé nabíjení objemu během prvních týdnů.

Reprodukce obrázků: screenshoty odpovídají výsledkům z dashboardu spuštěného nad projektem Example_05. Jednotlivé konfigurace vznikají ze slovníku `ParameterGrid` ve skriptu `examples/Example_05/run_example.py` a ukládají se do PostgreSQL projektu. Dashboard spuštěný přes `examples/Example_05/view_project.py` umožňuje vybrat úlohu v tabulce, zobrazit časové řady sond a porovnávat varianty podle geometrických parametrů, počtu patron, uložené energie, tepelných toků a účinností.

Co všechno se příklady ověřuje

Sada příkladů pokrývá hlavní funkcionality balíčku v rozsahu, který je relevantní pro funkční testování:

Funkcionalita	Pokryto v
Procedurální 2D-axisymetrická geometrie (Gmsh)	Example_01
Import geometrie z CAD (.step) + extrakce osově symetrie	Example_02, Example_03, Example_04
Plně 3D periodická geometrie (válcová výseč)	Example_05, Example_06
Term <code>UniformHeatSource</code> + <code>AmbientCooling</code>	Example_01, Example_02
Nelineární zdroj závislý na teplotě ($\sigma(T)$)	Example_03 (<code>SerialWiresResistiveHeating</code>)
PID regulátor jako Term s implicitní iterací	Example_04 (<code>PIDControlledHeatSource</code>)
Ko-simulace se zjednodušeným modelem budovy	Example_05, Example_06 (<code>HallC3</code>)
Stahování meteorologických dat (PVGIS / NSRDB)	Example_05
Parametrická studie (<code>ParameterGrid</code> , KDE řazení)	Example_05
PostgreSQL fronta úloh + MPI workery	Example_05
Webový dashboard s autentizací a RBAC	Example_05 (<code>view_project.py</code>)
Checkpointy a obnova běhu (<code>adios4dolfinx</code>)	Example_05, Example_06
Adaptivní časový krok podle $\max \Delta T $	všechny příklady
Sondy do CSV i do databáze	Example_01--04 (CSV), Example_05 (DB i CSV)

Každý příklad lze spustit samostatně a porovnat výstupní CSV (`results/unsteady.csv`) s referenčními hodnotami. Regrese se typicky projeví buď přímo ve fyzikálních sondách (změna zdroje, materiálu nebo okrajové podmínky), v čítačích `NLS_iter_step` / `KSP_iter_step` (změna chování řešiče), nebo v agregovaných veličinách `H_storage` a `Eff_*` na úrovni celého systému.

16 Časté problémy

Projev	Kam se dívat
SNES / KSP nedoběhne	Zmenšete <code>dt_max</code> , zpřísníte <code>dt_ctrl_interval</code> a zkontrolujte <code>T0 / T_guess</code> , okrajové podmínky i materiály.
Worker hlásí prázdnou frontu	Zkontrolujte <code>--project</code> , dostupnost databáze a stav úloh (COMPLETED, RUNNING apod.).
Duplicitně odmítnutá úloha	Úloha má stejnou SIGNATURE. Změňte parametry <i>nebo</i> zdrojový kód tak, aby kombinace dala jiný hash.
PVGIS selže	Častou příčinou je firewall, timeout nebo špatné souřadnice. Zvyšte <code>timeout</code> ve <code>fetch_hourly</code> ; cache lze smazat ručně.
Nedostatek místa	Zkontrolujte <code>checkpoint_dir</code> , binární sondy v databázi a <code>local_temp_dir</code> na výpočetních uzlech.
Chyba konfigurace (merge_configs selhává)	Spusťte z adresáře, kde leží správný <code>config.yaml</code> . Nepřidávejte klíče mimo <code>base_config.yaml</code> .
Po obnově z checkpointu chybí XDMF časy	Pokud při původním běhu <i>nebyl</i> XDMF zapnutý, snapshot v checkpointu neexistuje a soubor se otevře v režimu "w".
ParaView ukazuje jen mesh, žádné T	Otevřete <code>.xdmf</code> , ne <code>.h5</code> . Případně byl proces ukončen před prvním XDMF zápisem (čas <code>dt_xdmf</code> ještě nenastal).

Tento dokument popisuje stav repozitáře v době psaní. Pokud by se návod a zdrojový kód rozcházely, přednost má zdrojový kód.